

Mobile Apps with Expo

React Native, Expo, Expo Go, Expo Router



Svetlin Nakov, PhD

Co-founder @ SoftUni

Agenda

1. **Intro to Mobile App Development:** platforms, technologies, frameworks
2. **React Native and Expo:** React Native, Expo, Expo Go, mobile / Web render
3. **Initializing an Expo Project:** create and run an Expo app in VS Code
4. **Core UI Components in React Native:** native components, sample app
5. **Styling in React Native:** StyleSheet API, Flexbox, SafeAreaView, sample app
6. **Navigation with Expo Router:** file-based routing in Expo, screens and navigation, stack navigation, tabs navigation, routes and params
7. **Data Fetching and Backend:** CRUD operations with Supabase
8. **Exercises: Expo with Supabase:** food tracker app, task list app
9. **Publishing Expo App to Android / Apple App Store**



Sli.do Code

AppsAI

Join at
slido.com
#AppsAI



Breaks

20:00 / 21:00



Mobile App Development

Platforms, Mobile Technologies, UI Principles,
Mobile App Technologies, Mobile Back-End



Mobile App Platforms

- Main **mobile platforms**:
 - **Android** (tablets, smartphones)
 - **iOS** (tablets, smartphones)
- Different **screen sizes**, resolutions, pixel densities
- Different platform **conventions** and **UI expectations**
- Apps can target:
 - **Smartphones**
 - **Tablets**
 - **Foldable** devices
 - **Web** (as companion experience)



Mobile App Technology Choices

- **Native Android** apps
 - Kotlin / Java + Android Studio → best Android integration
- **Native iOS** apps
 - Swift / Objective C + XCode → best iOS integration
- **Cross-platform** frameworks
 - **React Native** + **Expo**: fastest path for React devs
 - **Flutter** + **Dart**: build once, run everywhere
- **Web-based** multi-platform apps:
 - **PWA** (Portable Web Application): fastest path for Web devs
 - Fastest Web app reuse, weaker native integration

Native vs. Cross-Platform

- **Native iOS / Android** apps:

- Maximum platform access
- Platform-specific codebases
- Higher cost for dev + maintenance

VS

- **Cross-platform** apps:

- Shared codebase
- Faster delivery
- Still can access native capabilities
- Lower control on device

- **React Native + Expo:**

- Reuse your React skills in the mobile world
- Production apps with quick development loop

Mobile UI / UX Principles

- **Touch-first UI:**
 - Big tap targets
 - Fewer dense controls
- **Small-screen design:**
 - Vertical layouts
 - Clear information hierarchy (size, spacing, and contrast)
- **Platform guidelines:**
 - [Apple Human Interface Guidelines](#)
 - [Google Material Design](#)
- **Mobile-first concerns:**
 - Safe areas (always visible)
 - Soft keyboard (don't hide inputs)
 - Offline states
 - Network latency

Mobile Back-End Options

- **BaaS platforms:**
 - Supabase
 - Google Firebase
 - Faster to build, but less control
- **Custom back-end:**
 - Node.js + Express
 - Next.js API routes / server actions
 - More effort, more control
- Common mobile **back-end services:**
 - Database, authentication, file storage, data sync, push notifications, long-running jobs, monitoring, data backup

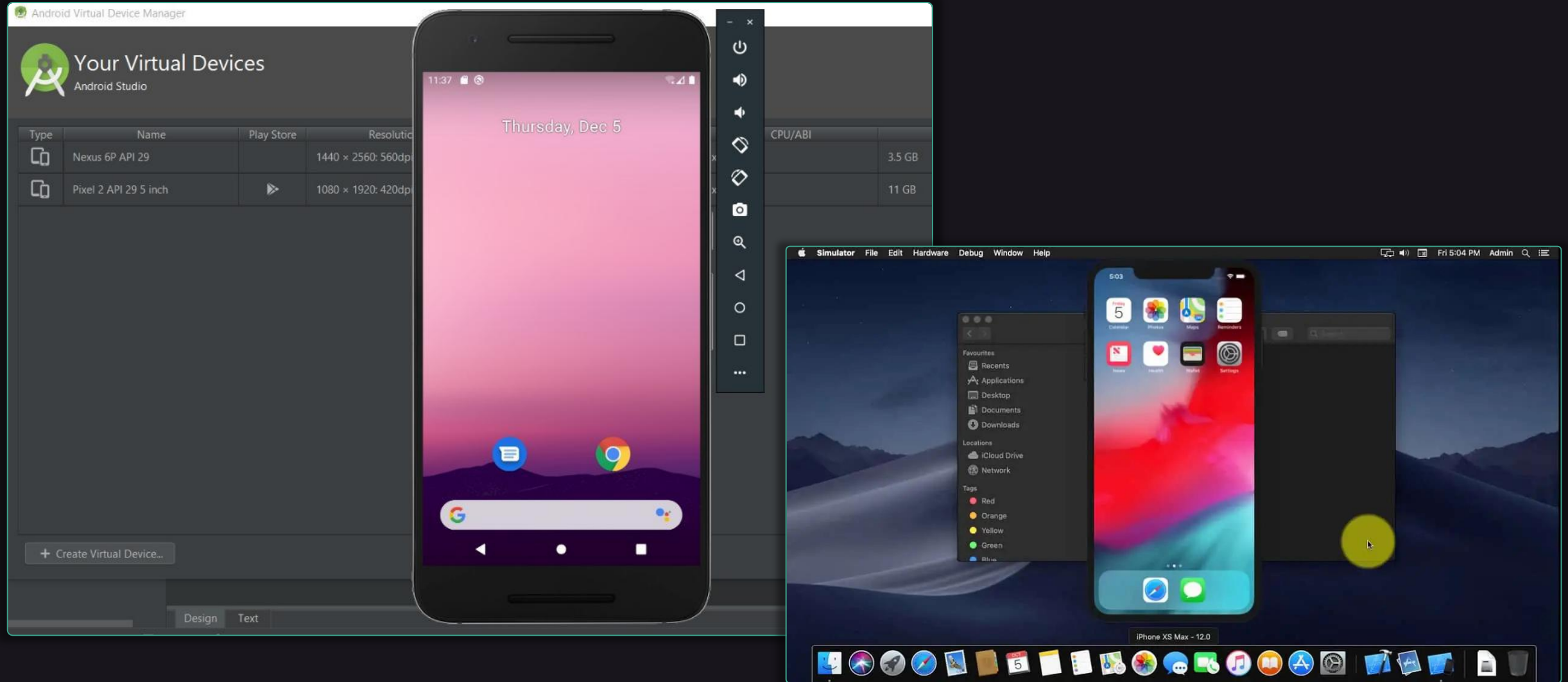
Physical Devices vs. Emulators

- **Physical devices:**
 - Real performance
 - Real gestures, cameras, sensors
 - Real OS behavior
- **Emulators / simulators:**
 - Quick iteration
 - Easy screenshots and debugging
 - Multiple devices on the same machine
- **Best practices:**
 - Build on emulator / simulator
 - Validate on real device before release

Local and Cloud Device Testing

- **Local emulators:**
 - Android Studio Emulator
 - LDPlayer (Android in Windows)
 - iOS Simulator (macOS)
- **Cloud-based device farms:**
 - BrowserStack
 - Firebase Test Lab
- **Useful for:**
 - Device coverage
 - Regression testing
 - Edge-case validation
- **App image** build services:
 - Expo EAS cloud workflows

Android Emulator / iOS Emulator



Browser Stack: Cloud for Devices



BrowserStack Live

Products

Invite team

AI-powered manual testing workspace with Live and 15+ other tools, for 10x faster testing.

Favourites (10/12)

Real Devices (33)

Apple iOS

Android

Windows

Mac

No SIM

SIM

PRO

Samsung

Google

OnePlus

Motorola

Xiaomi

Vivo

Oppo

Huawei

Realme

Galaxy S25

Galaxy S23

Galaxy S23

Galaxy S22

Galaxy S22+

Galaxy S23 Ultra

Galaxy S21

Galaxy S21

Galaxy S24 Ultra

Galaxy S24

Galaxy S26

Galaxy S26 Ultra

Galaxy S24

Galaxy A16 5G

Galaxy S25 Ultra

Galaxy Z Fold 7

15

13

13

12

12

13

11

12

14

14

16

16

16

15

15

16

YOUR FREQUENTLY USED

Galaxy S21+

iPhone 12

iPhone 12 Mini

iPhone 12 Pro Max

11.0

14.0

14.0

14.0

TRENDING

iPad Pro 12.9 2021

iPhone 12 Pro

iPhone 11

iPhone 11

iPhone 8

14.0

14.0

13.0

14.0

13.0

NEW DEVICES

iPhone XS

Galaxy S20

14.0

10.0

View All

Schedule UI monitoring

Switch browser

Local Testing

Zoom

Report a bug

DevTools

Settings

Rotate device

Change Location

Throttle Network

Galaxy S21 v11.0

Screen Reader

Stop Session

Local Settings

General

Resolve all URLs through my network

Use my local IP for Google Analytics tracking

Folder Testing

This feature is currently not supported through the Local Desktop app.

Learn more about Local Testing

13:34

browserstack.com

BrowserStack

Testing

Contact us

Test your websites

LIVE

You are testing on a

Emulator Real Device!

Serial Number

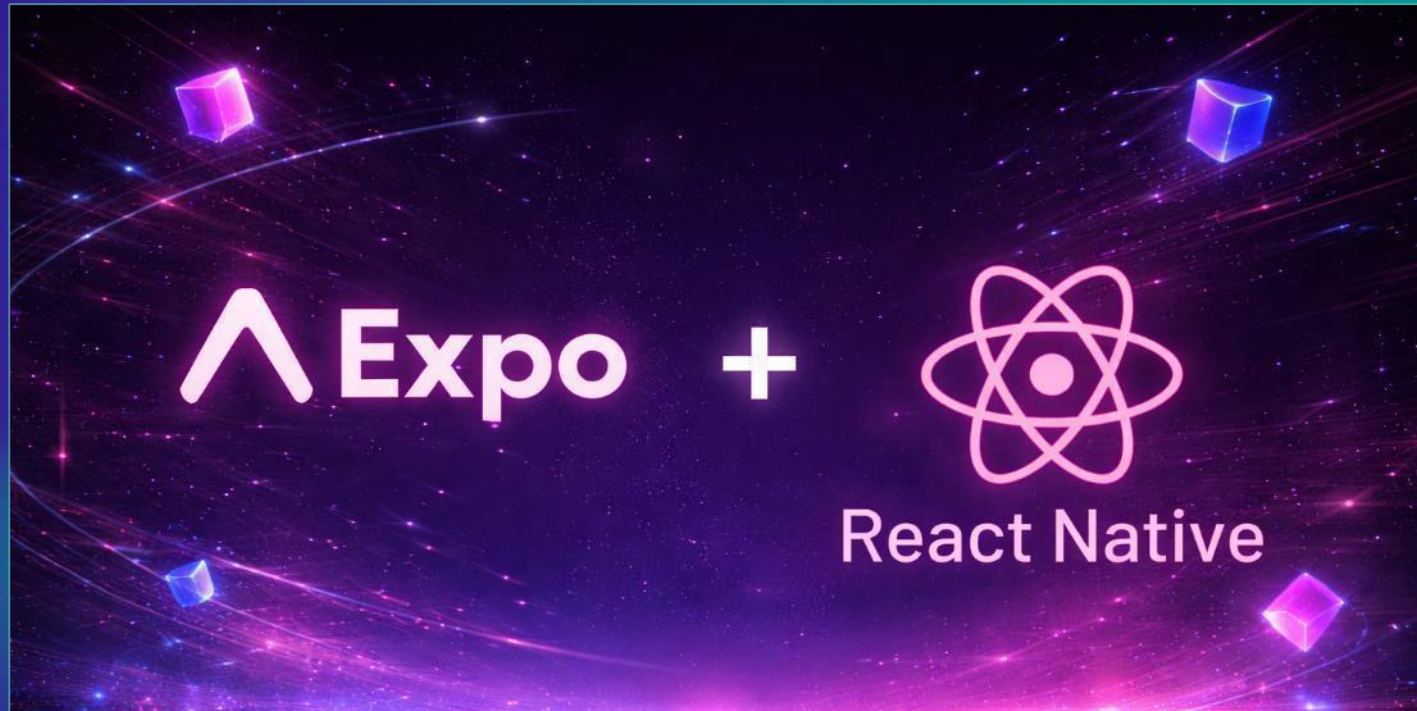
Purchase Date

R5CNCOMXXXX

5th Feb, 2021

React Native and Expo

React for Mobile, Native UI Rendering,
Expo Toolchain, Expo Go



What is React Native?

- **React Native** == mobile app dev framework, based on JavaScript + React
 - Write **React components** in JS / TypeScript
 - Render **native mobile UI** (Android / iOS) instead of HTML, CSS and DOM
- Reuse well-known React concepts:
 - **JSX, state, props, hooks**, component composition
- Different rendering target (Android / iOS):
 - **Mobile UI elements**, not browser tags



What is Expo?

- **Expo** == **platform** + **toolchain** for universal React apps
 - Speeds up project setup and development of React Native
- Expo components:
 - **Expo SDK** (JS APIs for mobile)
 - **Dev server** (run apps with fast refresh)
 - **Expo Go** (run apps on physical device)
 - **Build and submit services** (publish to app store)
- Excellent ecosystem for JS & React developers



React for Web vs. React Native

- **React for Web:**

`<div>, <p>, <button>,
<input>, `

- CSS files
- Browser DOM events
- Runs in a Web browser

- **React Native:**

`<View>, <Text>, <Pressable>,
<TextInput>, <Image>`

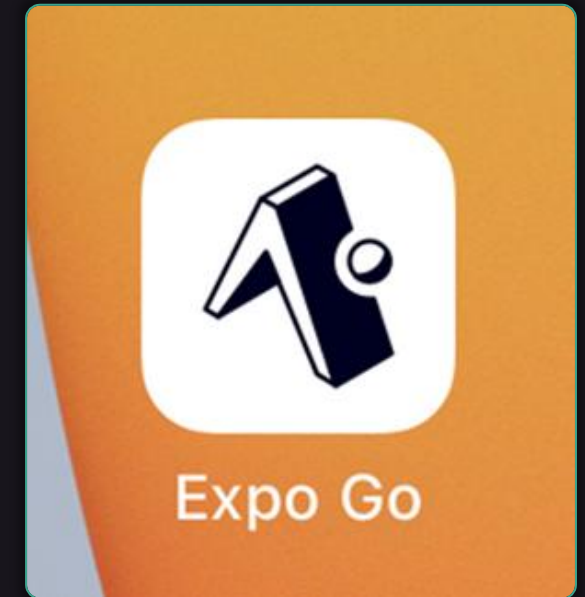
- StyleSheet / style objects
- Native touch events
- Runs in a mobile device

VS

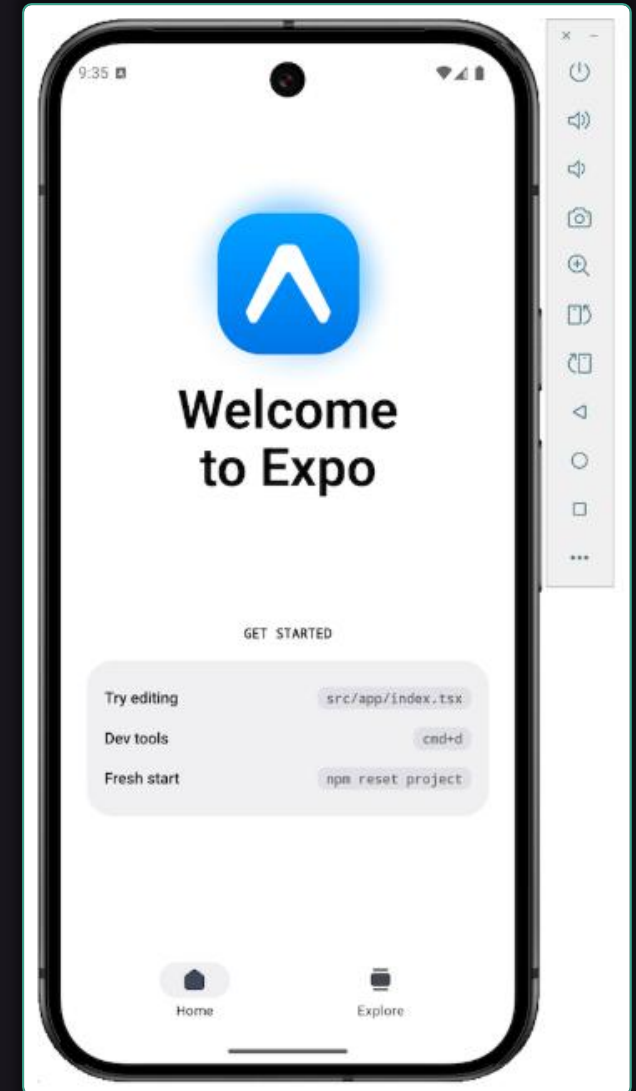
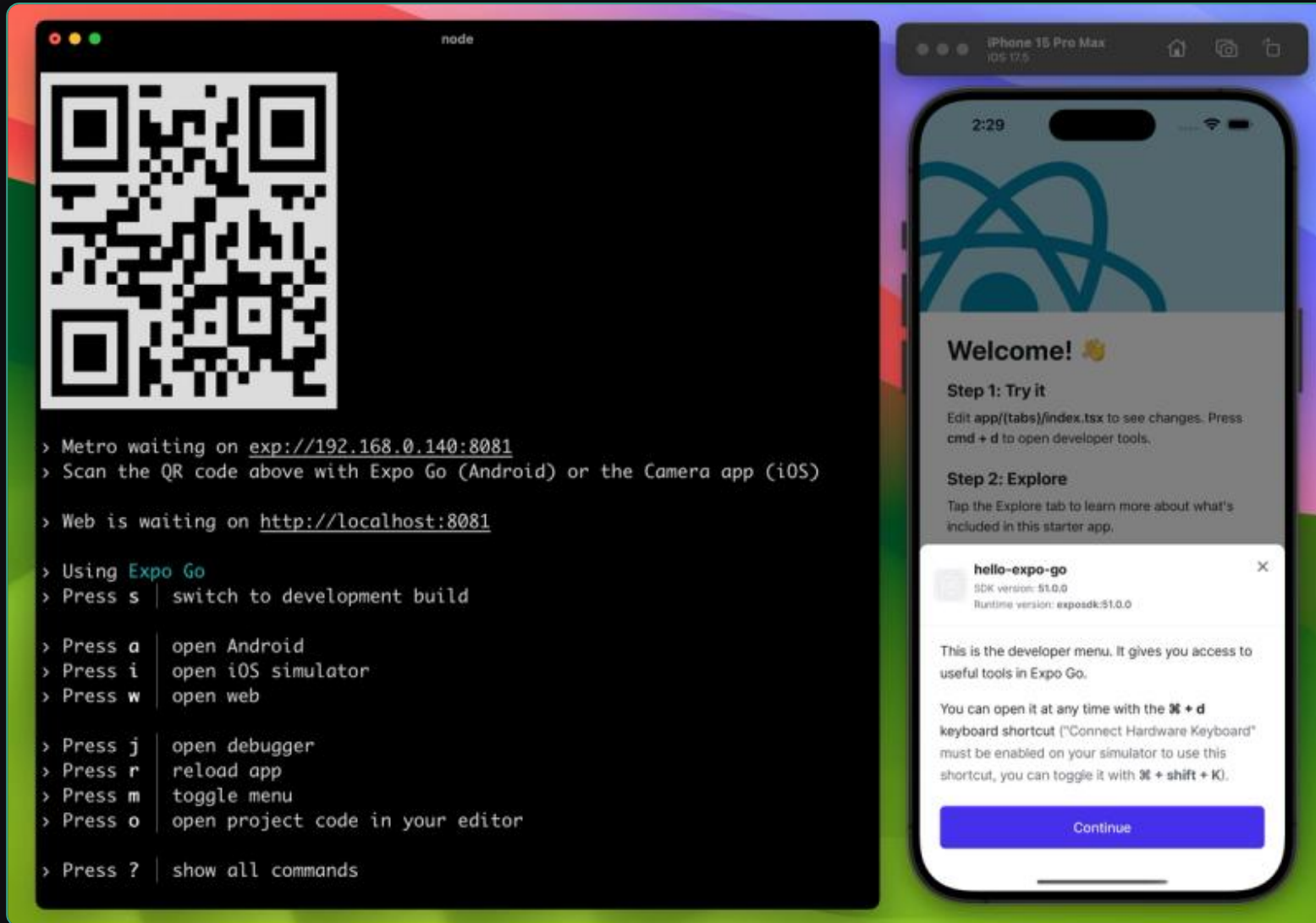
- **React thinking stays the same:** UI building blocks change

Expo Go (Expo App Runner)

- **Mobile app** for running Expo projects in dev mode
 1. Install **Expo Go** app on Android / iPhone
 2. Start Expo project locally (on a laptop):
npm run web
 3. **Scan QR code** to instantly test on device
- No app store publishing required
- **Fast feedback loop:**
 - Edit code → see changes on device immediately



Expo Go – Screenshots



Why React Native + Expo for AI-First Dev?



- **React Native + Expo == AI-friendly** platform
 - **JavaScript** is very productive for AI dev agents
 - Familiar **React patterns**: components, hooks, props, state
 - **AI agents** can scaffold screens and logic quickly
 - Great for prompt-driven prototyping / development
- Expo **removes native setup friction**
 - Faster setup, faster development cycle, faster shipping
 - No need for emulators, easy to test on a real device

Initializing an Expo Project

Using `create-expo-app`, TypeScript Setup, Running on Web and Devices



Creating an Expo App

- Creating a new **Expo + TypeScript** app:

```
npx create-expo-app@latest .
```

- Choose the **TypeScript** template
 - Creates a **Node.js** project folder
 - Installs **dependencies** + starter files
 - Creates a ready-to-run **Expo project**
- Common run targets:

```
npm run web
```

```
npm run android
```

```
npm run ios
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS Expo-mobile-apps> npx create-expo-app@latest mobile-app
Creating an Expo project using the default template.

To choose from all available templates (https://github.com/expo/expo/tree/main/packages/create-expo-app)
use the --template arg:
$ npx create-expo-app --template

To choose from all available examples (https://github.com/expo/examples)
$ npx create-expo-app --example

✓ Downloaded and extracted project files.
> npm install
```

Bootstrapping Expo Project: Live Demo

- **Steps** to bootstrap an Expo project:

1. Create Expo project:

```
npx create-expo-app@latest .
```

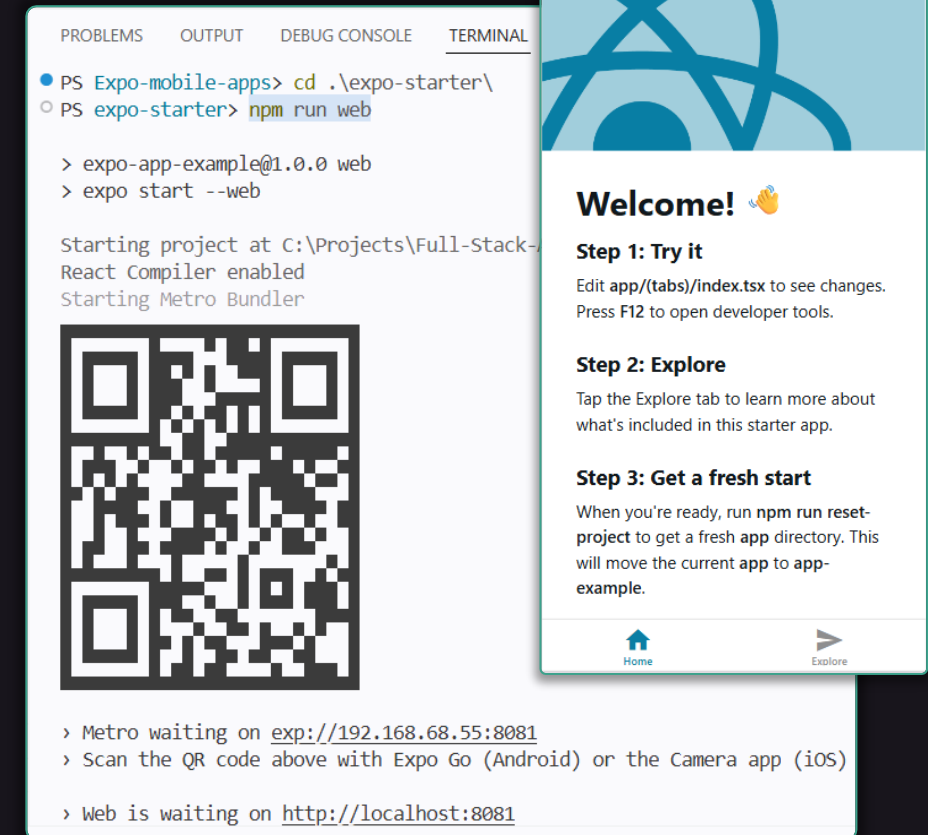
2. Run in a browser: `npm run web`

- Scan the **QR code** with **Expo Go**

3. Run as native Android app:

```
npm run android
```

4. Edit the welcome text and verify hot reload



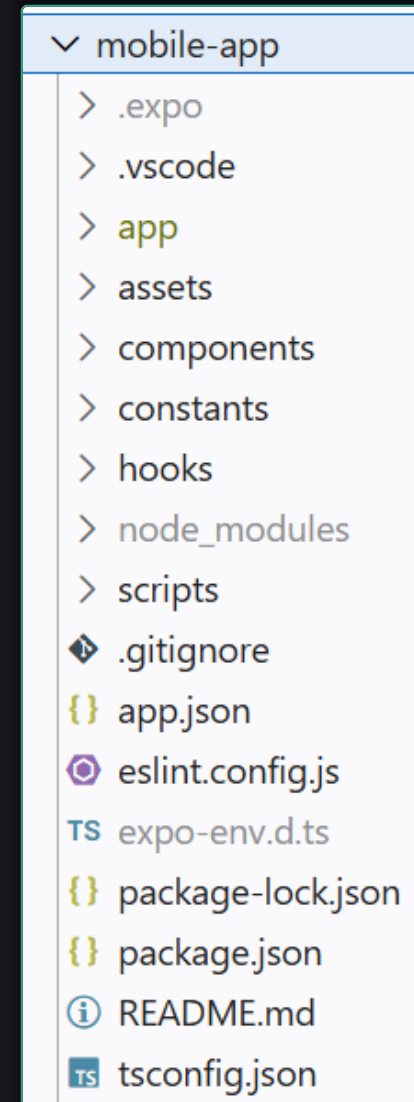
The App in a Web Browser / Device



- **Web:**
 - Fast for layout / code logic checks
 - Easy browser dev tools
 - Instant hot reload
 - Test via Web browser:
<http://localhost:8081>
- **Device:**
 - Real keyboard / touch behavior
 - Real platform feel
 - Test via **Expo Go** with QR code (instant hot reload)
 - Or install as native app
- **Use Web** during development, test also on device

Expo Project Structure: Overview

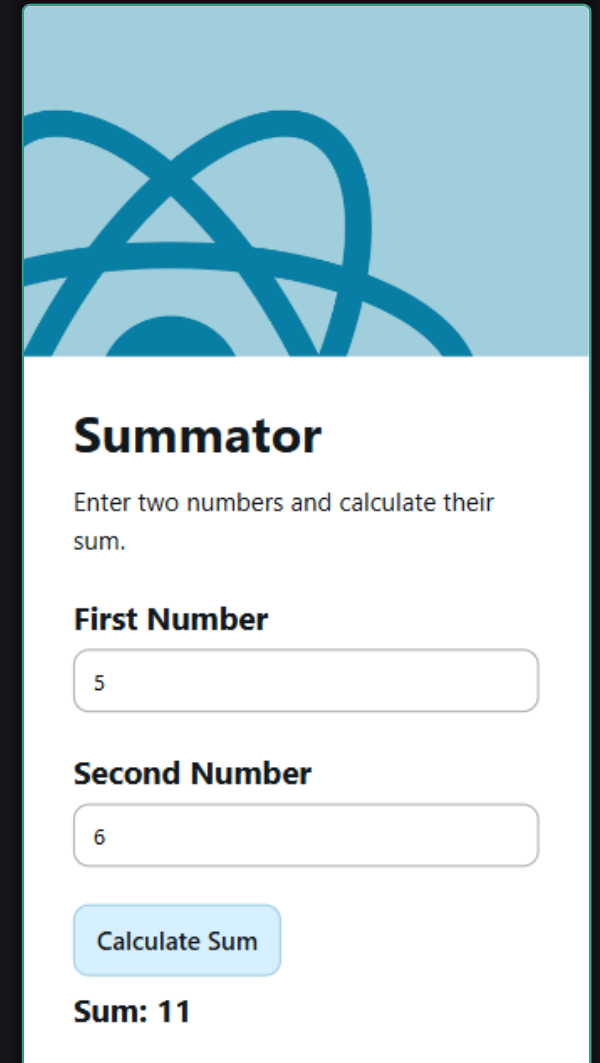
- Expo project key **folders** and **files**:
 - **app/** — screens and routes
 - **components/** — reusable UI pieces
 - **assets/** — images and icons
 - **package.json** — scripts and dependencies
 - **tsconfig.json** — TypeScript settings
 - **app.json** — app name, slug, icons, splash, bundle config



Summator Screen

- Implement custom functionality (screen) in an Expo app:
 - **Summator** (enter two numbers and calculate their sum)
- AI prompt:

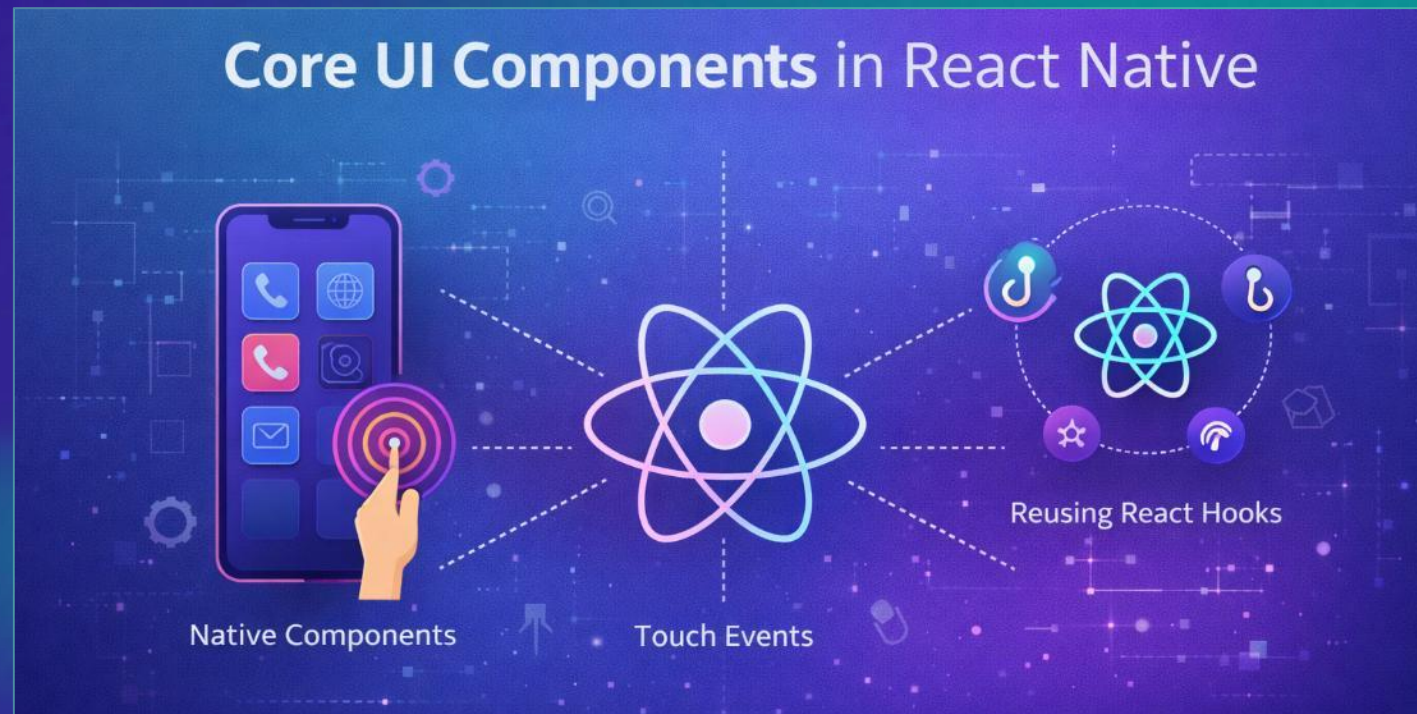
Implement **summator** screen (**summator.tsx**): enter two numbers and calculate their sum



The image shows a mobile app interface for a summator. It has a light blue header with a stylized atom logo. Below the header, the title "Summator" is displayed in bold. Underneath the title, there is a prompt: "Enter two numbers and calculate their sum." There are two input fields: "First Number" with the value "5" and "Second Number" with the value "6". Below these fields is a blue button labeled "Calculate Sum". At the bottom, the result "Sum: 11" is displayed.

UI Components in React Native

View, Text, TextInput, Image,
ScrollView, Pressable, Hooks Reuse



The Most Important Building Blocks

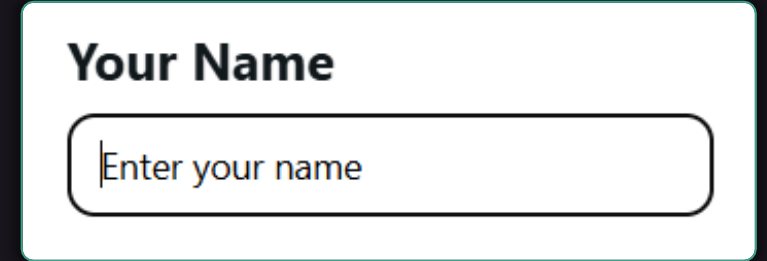
- **View** — layout container (panel)
- **Text** — render text content (static text)
 - Text must always live inside a **Text** component — not directly in **View**
- **TextInput** — user input (text box)
- **Image** — local or remote image
- **ScrollView** — vertical / horizontal scrolling area
- **Pressable** — touch interactions (button)

```
<View>  
  <Text>First name:</Text>  
  <Text>Last name:</Text>  
</View>
```

```
<TextInput  
  placeholder="First name"  
  value={firstName}  
  onChangeText={setFirstName} />
```

TextInput for User Data

- Use **<TextInput />** for typing
 - Controlled input with **useState()**
- Useful props:
 - **value, onChangeText, placeholder, keyboardType**
- **Example:**



```
const [name, setName] = useState("");  
<TextInput value={name} onChangeText={setName} />
```

- **<Pressable>** handles tap interactions: buttons, cards, rows

Handling Touch Events

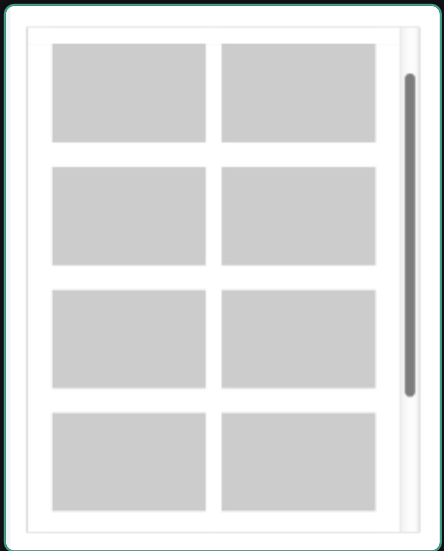
- Use **<Pressable>** for touch events
 - **Pressable** handles tap interactions
 - Can style based on pressed state using **style** callback
 - Common use: buttons, cards, action rows
- Example:



```
<Pressable onPress={saveItem}>  
  <Text>Save</Text>  
</Pressable>
```


ScrollView and FlatList

- **<ScrollView>** and **<FlatList>** display scrollable content
 - **<ScrollView>**: simple scrolling, renders all children at once
 - **<FlatList>**: efficient rendering for large collections, with **lazy loading** (loads only what is visible on the screen)



- Short **static content** (e.g. a form with fields)
 - Use **<ScrollView>**
- **Data collections** (e.g. data list / table)
 - Use **<FlatList>**

React Hooks in Mobile Components

- **React hooks** are reused in React Native
 - **useState()** — component state management
 - **useEffect()** — side effects, data fetching, subscriptions
 - **useMemo()** — memorize expensive computations
- New platform, same **React mental model**
 - Biggest change: **native UI components** instead of browser elements; no CSS — only **JavaScript styles**
 - Mobile-specific **APIs**: Platform, Dimensions, Keyboard

Metric Converter

Convert values between metric and imperial units.

From Unit

mm

cm

m

km

inch

foot

To Unit

mm

cm

m

km

inch

foot

Convert

Result: 500 cm

Live Demo: Metric Converter

Building a Metric Converter
in React Native + Expo

Building a Metric Converter

- Build a **Metric Converter** screen
 - Units: mm, cm, m, km, inch, foot
 - Selecting source and target units

Implement **Metric Converter** screen (**metric-converter.tsx**): select value + source unit + target unit, convert the value and display the result.

Units: mm, cm, m, km, inch, foot.

Metric Converter

Convert values between metric and imperial units.

From Unit

mm cm **m** km inch foot

To Unit

mm **cm** m km inch foot

Convert

Result: 500 cm

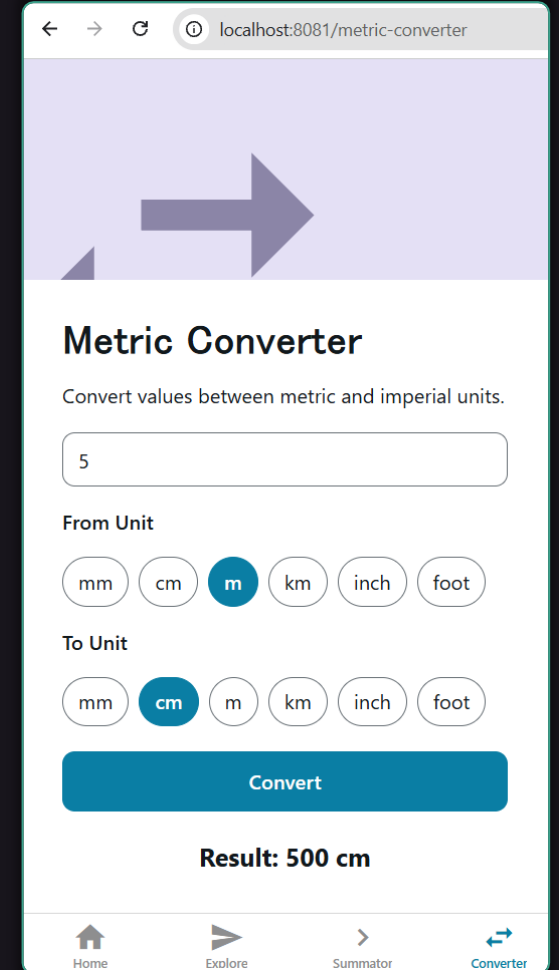
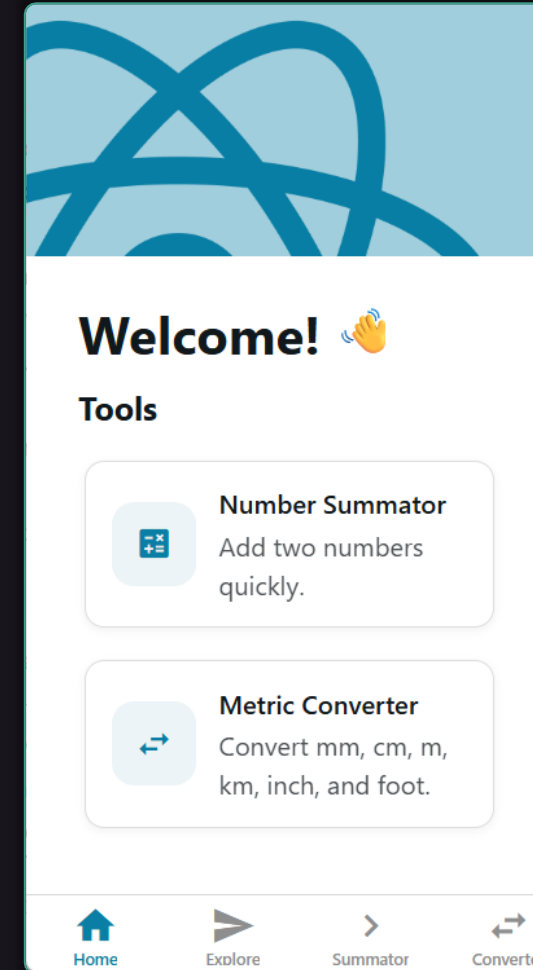
Adding a Link to the Converter

- Adding navigation entry point

Display **card links** on the **home screen**: Number Summator and Metric Converter.

Display **tabs** in the **app layout**: Summator and Converter tabs.

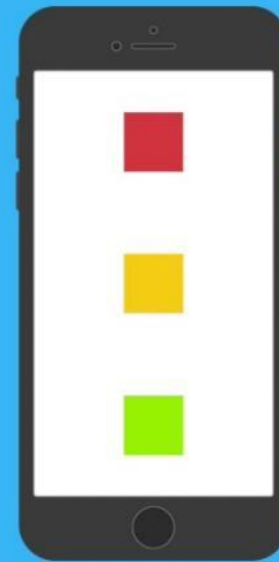
Style the summator and converter screens with the same **visual pattern** as the other screens (with large icon at the top).



Styling in React Native

StyleSheet API, Flexbox Layout,
Safe Areas, Mobile Spacing

```
render() {  
  return(  
    <ParentView style={styles.parentView}>  
      <redBlock/>  
      <yellowBlock/>  
      <greenBlock/>  
    </ParentView>  
  )  
}  
}  
  
const styles = StyleSheet.create({  
  parentView: {  
    flex:1,  
    flexDirection: 'column',  
    justify  
  },  
})
```



Using the StyleSheet API

- Styling with **StyleSheet**
 - Styles are **JavaScript objects** — no CSS files
 - Property names use camelCase, e.g. **backgroundColor, fontSize**
 - Units are numeric, e.g. **padding: 12** (not "12px")
- Sample styles definition:

```
const styles = StyleSheet.create(  
  { container: { flex: 1, padding: 16 } }  
);
```

Benefits of the StyleSheet API

- Benefits of the **StyleSheet API**:
 - React **components** and their **styles** stay together
 - **Autocomplete** in TypeScript works very well
 - **No CSS selectors** — styles are scoped to components
 - **No conflicting styles** (with the same CSS selector)
- Best practice in larger apps:
 - Extract **design system components** as building blocks
 - Example: **<AppInput>** instead of **<TextInput>**

Sample Design System Component

```
import { TextInput, StyleSheet }
from 'react-native';

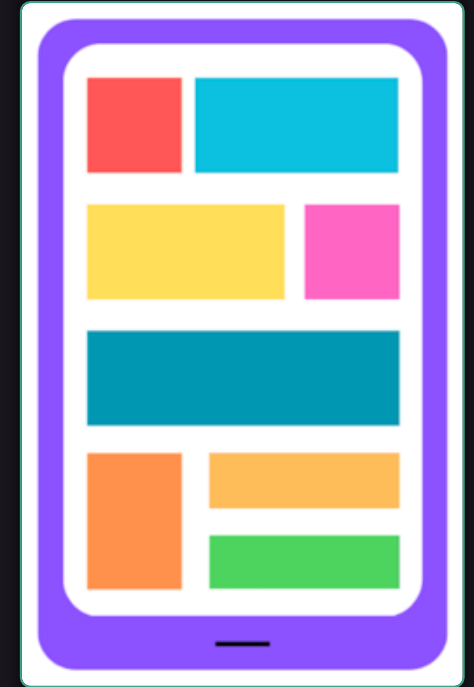
export function AppInput(
  { style, ...props }) {
  return (
    <TextInput
      style={ [styles.input, style] }
      placeholderTextColor="#999"
      {...props}
    />
  );
}
```

```
<AppInput placeholder="Surname" />
<AppInput placeholder="Name" />
```

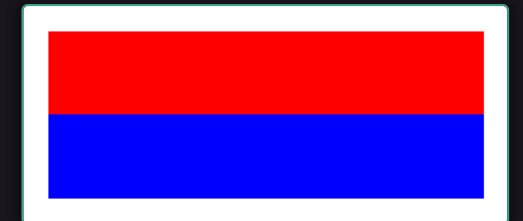
```
const styles =
StyleSheet.create({
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    padding: 12,
    borderRadius: 8,
    marginBottom: 12,
    fontSize: 16,
  },
});
```

Flexbox in React Native

- **Flexbox** is the main layout system
 - Default: **flexDirection: "column"** (unlike CSS default "row")
 - Common props: **flex, justifyContent, alignItems, gap, margins, padding**
 - Think in stacked mobile sections



```
<View style={{ flex: 1 }}>  
  <View style={{ height: 50, backgroundColor: 'red' }} />  
  <View style={{ height: 50, backgroundColor: 'blue' }} />  
</View>
```



Flexbox in React Native

- Common **styling mistakes**

- Designing like desktop, not mobile
- Too many items in one row
- Tiny fonts and tiny tap targets

```
<SafeAreaView  
  style={{flex: 1,  
    justifyContent: 'center',  
    alignItems: 'center' }}>  
  <Text>Centered</Text>  
</SafeAreaView>
```

- **Safe areas** matter

- Phones have notches, status bars, home indicator areas
- Use **<SafeAreaView>** to prevent content hiding behind system UI
- Especially important for top headers and bottom actions

Global Styles

- Components can share **global styles (globalStyles.js)**:

```
import { StyleSheet }
from 'react-native';

export const globalStyles =
StyleSheet.create({
  container: {
    flex: 1, padding: 16,
    backgroundColor: '#fff',
  },
  title: { fontSize: 20,
    fontWeight: 'bold',
  },
});
```

globalStyles.ts

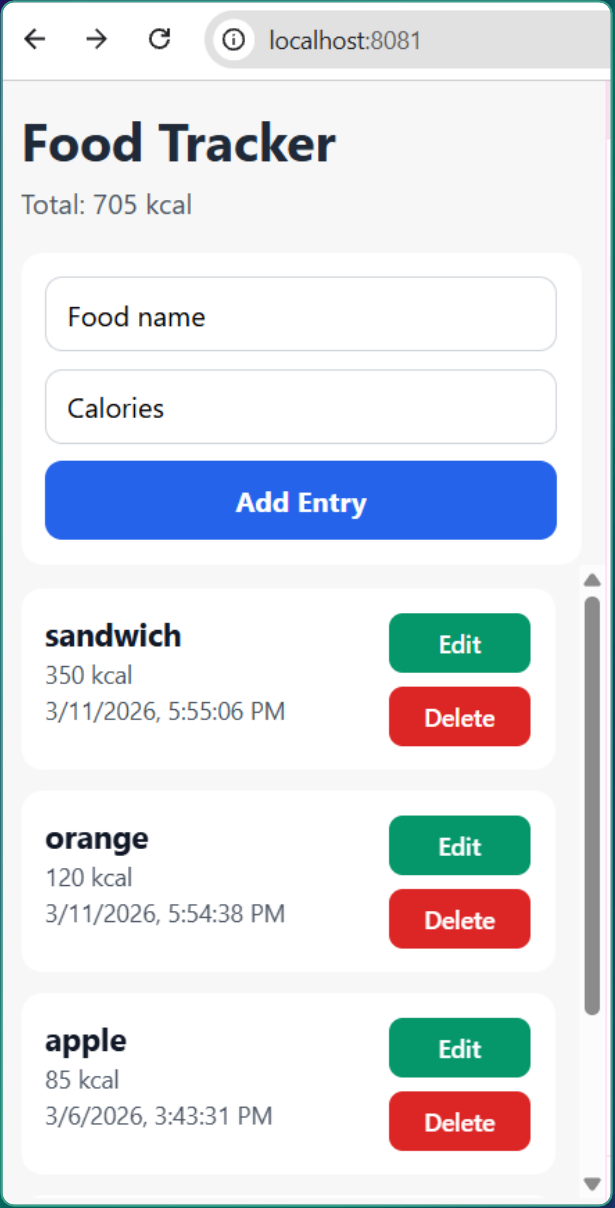
```
import { View, Text } from 'react-native';
import { globalStyles } from './globalStyles';

export default function AboutScreen() {
  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.title}>
        About Us</Text>
      </View>
    );
}
```

about.tsx

Food Tracker Mobile App

Local CRUD App with
Async Storage



The screenshot shows a mobile application interface for a food tracker. At the top, there is a browser-like address bar with navigation icons and the text 'localhost:8081'. Below this is the title 'Food Tracker' and a summary 'Total: 705 kcal'. The main form area contains two input fields: 'Food name' and 'Calories', followed by a blue 'Add Entry' button. Below the form is a list of three food entries, each with an 'Edit' button (green) and a 'Delete' button (red). The entries are: 'sandwich' (350 kcal, 3/11/2026, 5:55:06 PM), 'orange' (120 kcal, 3/11/2026, 5:54:38 PM), and 'apple' (85 kcal, 3/6/2026, 3:43:31 PM). A vertical scrollbar is visible on the right side of the list.

localhost:8081

Food Tracker

Total: 705 kcal

Food name

Calories

Add Entry

sandwich
350 kcal
3/11/2026, 5:55:06 PM

Edit Delete

orange
120 kcal
3/11/2026, 5:54:38 PM

Edit Delete

apple
85 kcal
3/6/2026, 3:43:31 PM

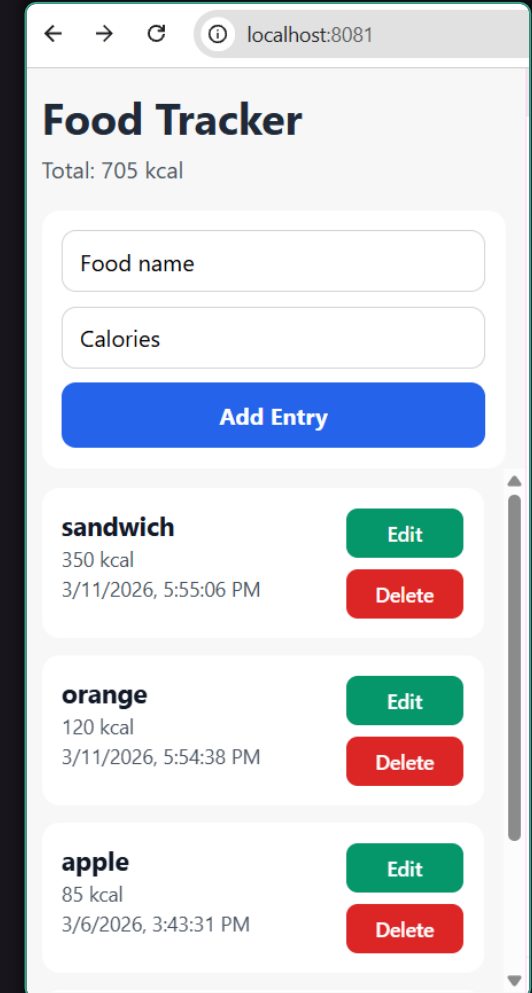
Edit Delete

Building a Food Tracker Mobile App

- Building a simple app with **Expo** and **TS**:

Build a simple **Food Tracker** mobile app with **Expo** and **TypeScript**

- **View list** of food log entries / **create** / **edit** / **delete** entry
- Data model: food name, calories, timestamp
- **Persist data** between app restarts
- Keep everything **local** – no auth, no database, no routing



Food Tracker App Implementation

- Food log **data type** (TypeScript)

```
type FoodLog = {  
  id: string;  
  foodName: string;  
  calories: number;  
  timestamp: string;  
}
```

- Suggested **UI structure**
 - Header with app title
 - Form for add / edit
 - Food logs list of cards
 - Action buttons: edit, delete

Async Storage

- What is **Async Storage** in Expo?
 - **Key-value storage** for **local** device persistence
 - Good for small app state, cached user data, simple prototypes
 - Not a relational database — perfect for first local CRUD app
 - In React Native for Web it uses **localStorage**
 - In Android / iOS it uses platform's internal storage



Async Storage

- Install Expo Async Storage

```
npx expo install @react-native-async-storage/async-storage
```

- Save data:

```
await AsyncStorage.setItem("foodLogs", JSON.stringify(logs))
```

- Read data:

```
let rawData = await AsyncStorage.getItem("foodLogs");  
const json = JSON.parse(rawData);
```

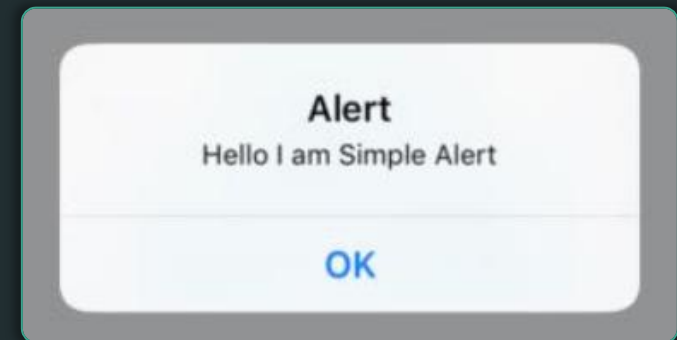
Implementing the App

- Building the app **step by step** (with several AI prompts):
 - Step 1: create a new **Expo + TypeScript app**
 - Step 2: **define data**: food log entry + food logs list
 - Step 3: create "Add food log" **form + state**
 - Step 4: render the food logs in a **<FlatList>**
 - Step 5: implement local persistence with **Async Storage**
 - Step 6: implement **[Edit]** action
 - Step 7: implement **[Delete]** with confirmation
 - Step 8: **polish** the UI and layout

Notes about Alerts

- **Android** and **iOS** support **native alert dialogs**: show a message / show a warning / confirm / cancel action
 - The built-in **Alert.alert(...)** API in React Native
 - Alerts API does **not work in React Web** → needs a fallback

```
function showAlert(title, message) {  
  if (Platform.OS === 'web')  
    window.alert(message);  
  else  
    Alert.alert(title, message);  
}
```



Navigation with Expo Router

File-Based Routing, Stack Navigation, Route Groups, Passing and Reading Parameters



Expo Router

- **Expo Router** – a routing system for Expo apps
- File-based routing in Expo
 - **app/index.tsx** → **/** (home route)
 - **app/details.tsx** → **/details** (static route)
 - **app/profile/[user].tsx** → dynamic route like **/profile/alex**
- Navigation to a view: **router.push("/dashboard")**
- Install and configure Expo Router
 - Expo Router comes with the default Expo template:
npx create-expo-app@latest

File-Based Routing in Expo

- Passing **route parameters**:

```
router.push({ pathname: "/details", params: { id: "42" } });  
const { id } = useLocalSearchParams();
```

- **Dynamic routes**, e.g. **/news/7**
 - **app/news/[id].tsx** — reuse one screen for many items
- **Route groups**: (tabs), (auth), (admin)
 - Organize screens logically without changing the URL path
 - Shared layout for each group: **_layout.tsx**

Route Groups – Example

```
app/  
  _layout.tsx  
  (tabs)/  
    index.tsx  
    settings.tsx  
  (auth)/  
    login.tsx  
    register.tsx  
  (admin)/  
    _layout.tsx  
    users.tsx
```

- The structure maps to these URLs:
 - (tabs)/index.tsx → **/**
 - (tabs)/settings.tsx → **/settings**
 - (auth)/login.tsx → **/login**
 - (auth)/register.tsx → **/register**
 - (admin)/users.tsx → **/users**

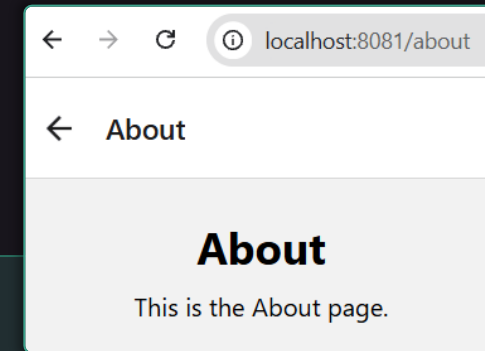
Navigation Between Screens

- **Navigate** between screens (pages)
 - **router.navigate("/")** — go to root
 - **router.push("/details")** — forward navigation
 - **router.back()** — return in stack flow
- Passing route **parameters**
 - Push with params:
router.push({ pathname: "/details", params: { id: "42" } })
 - Read params from a route:
const { id } = useLocalSearchParams<{ id?: string }>()
 - Used in [View Details] and [Edit] screens

Stack and Tabs Layouts

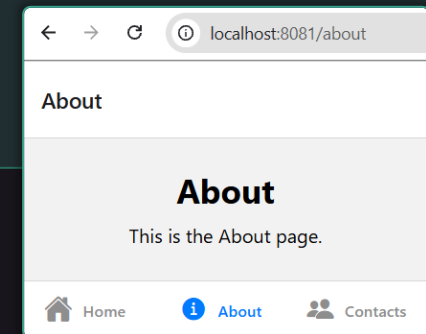
- **<Stack />** defines a **stack-based navigation system** for all routes in a folder (push / go back)

- Adds a **header (navbar)**



```
app/  
  _layout.tsx  
  index.tsx  
  about.tsx  
  contacts.tsx
```

```
// app/_layout.tsx  
import { Stack } from 'expo-router';  
  
export default function Layout() {  
  return <Stack />;  
}
```



- **<Tabs />** defines **bottom tabs navigation**

Expo Router – Example

- AI prompt to build **multi-screen** app with **stack navigation**:

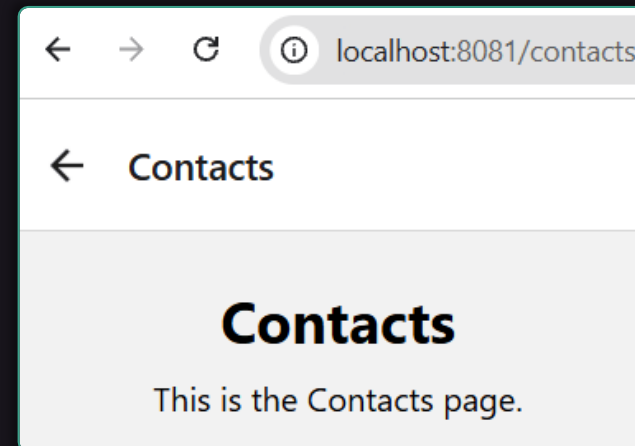
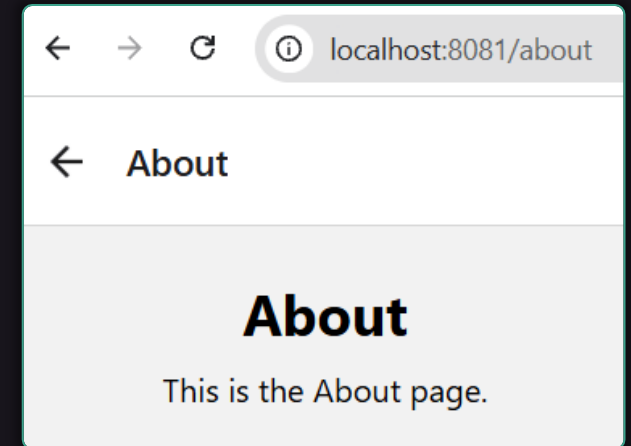
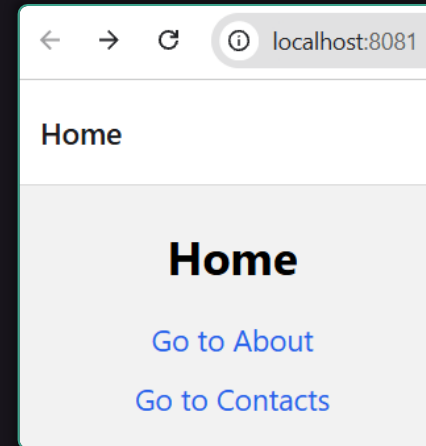
Build a **multi-screen** app
with Expo and Expo Router

Implement 3 screens:

Home | **About** | **Contacts**

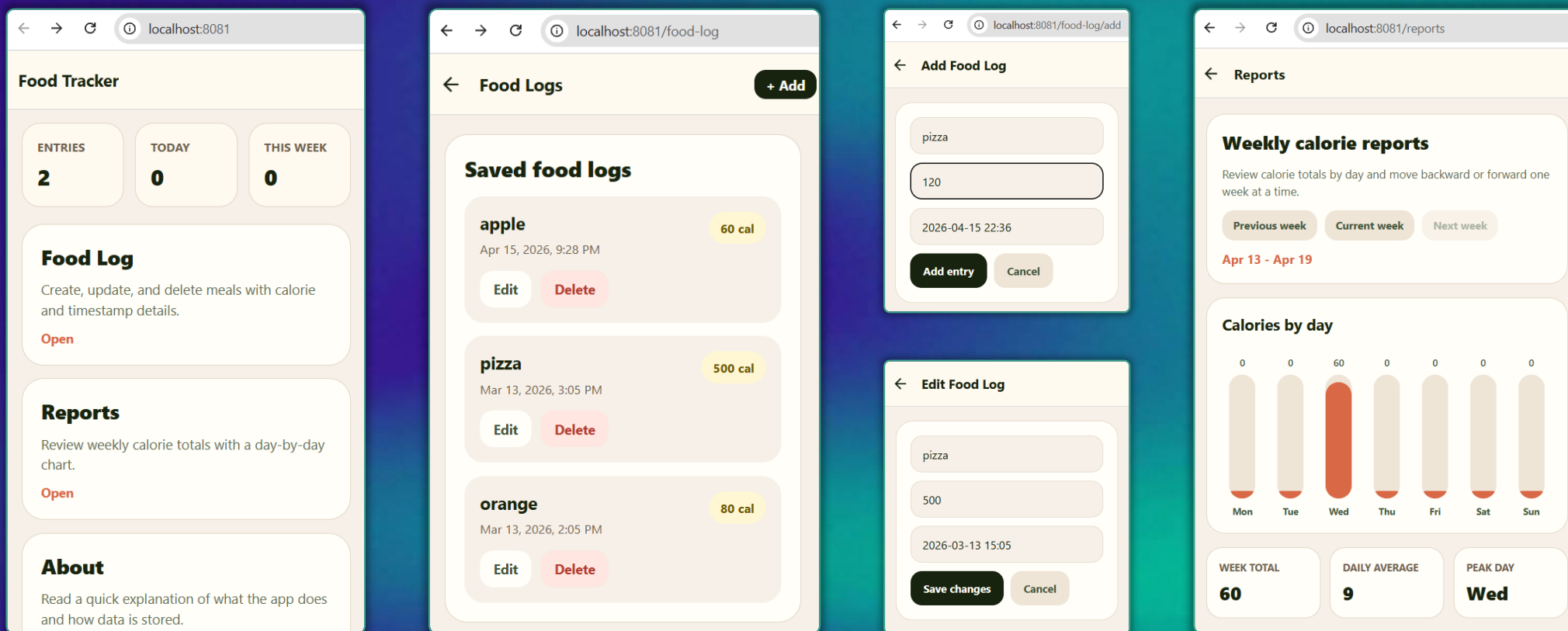
Implement layout with
<Stack />

At the **Home** screen show
links to About and
Contacts



Exercise: Multi-Page Food Tracker

With React Router and Async Storage



Exercise: Multi-Page Food Tracker

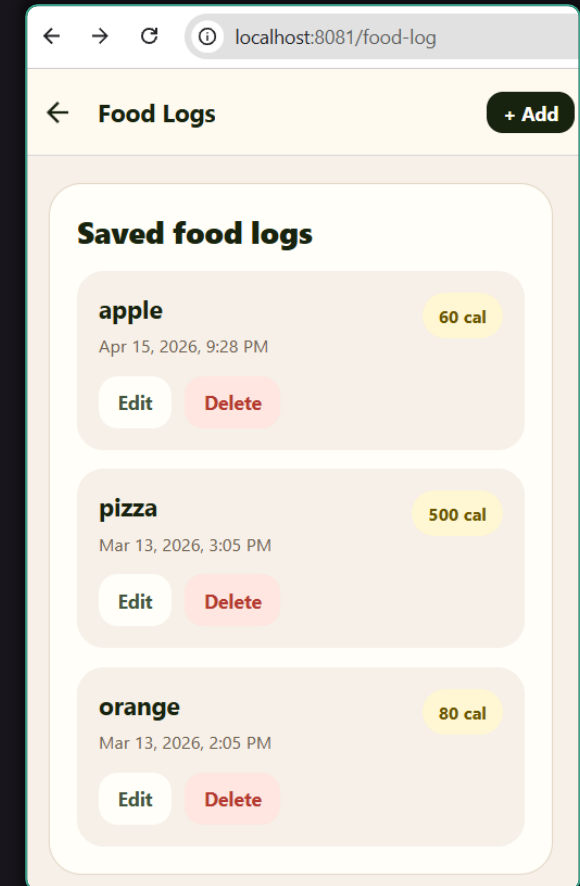
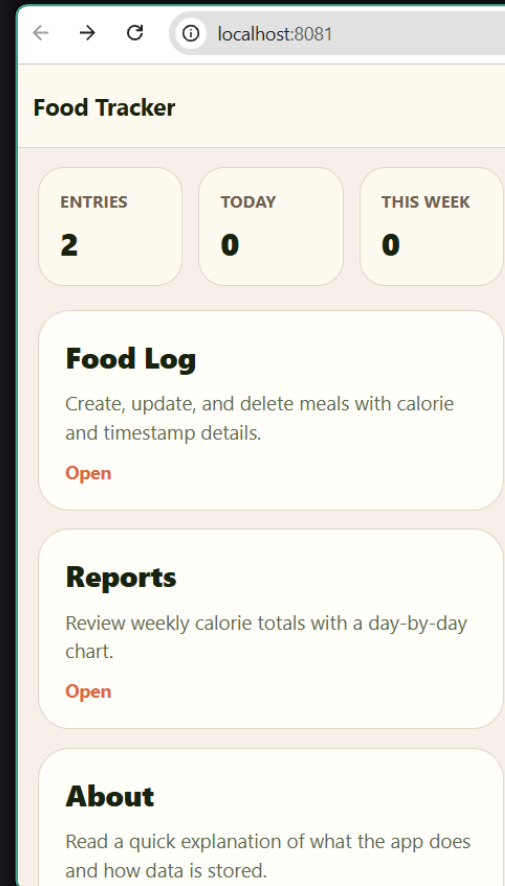
- Create a multi-page food tracker with React Router

Create a **multi-page "Food Tracker"** mobile app with **Expo**, **TypeScript** and **Expo Router**

- Food logs hold: id, food name, calories, timestamp
- Keep app data in a **local data storage** (Async Storage)
- **Screens**: Home screen, Food Log screen, Reports, About screen
- **Home** screen: links to Food Log, Reports and About screens
- **Food Log** screen:
 - Implement view, create, update, and delete (CRUD) functionality
 - Use separate screens for add and edit food log entry
- **Reports** screen: display a chart "Calories by day", week by week
- **About** screen: explain a few words about the app
- Implement **<Stack /> navigation** between screens

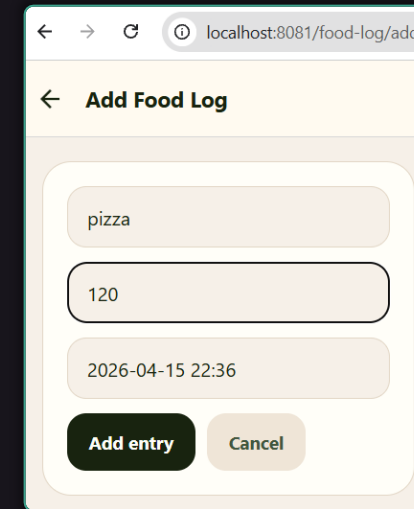
Home and Food Log Screens

- **Home** screen:
 - Brief app intro + quick stats (total entries, calories today)
 - Navigation links to Food Log, Reports, About
- **Food Log** screen:
 - Display all entries with edit / delete actions
 - Navigate to Add/Edit screens
 - Good use case for **FlatList**



Add/Edit Screen and Reports

- **Add / Edit screen:**
 - Reuse the same form for create and update
 - Read **id** from route params → load entry for edit mode
- **Reports screen:**
 - Show "Calories by day" — grouped totals, weekly view

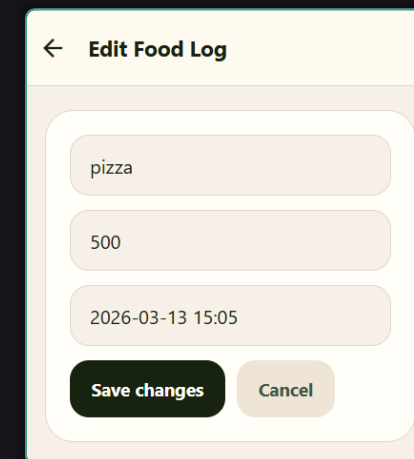


localhost:8081/food-log/add

← Add Food Log

Form fields: pizza, 120, 2026-04-15 22:36

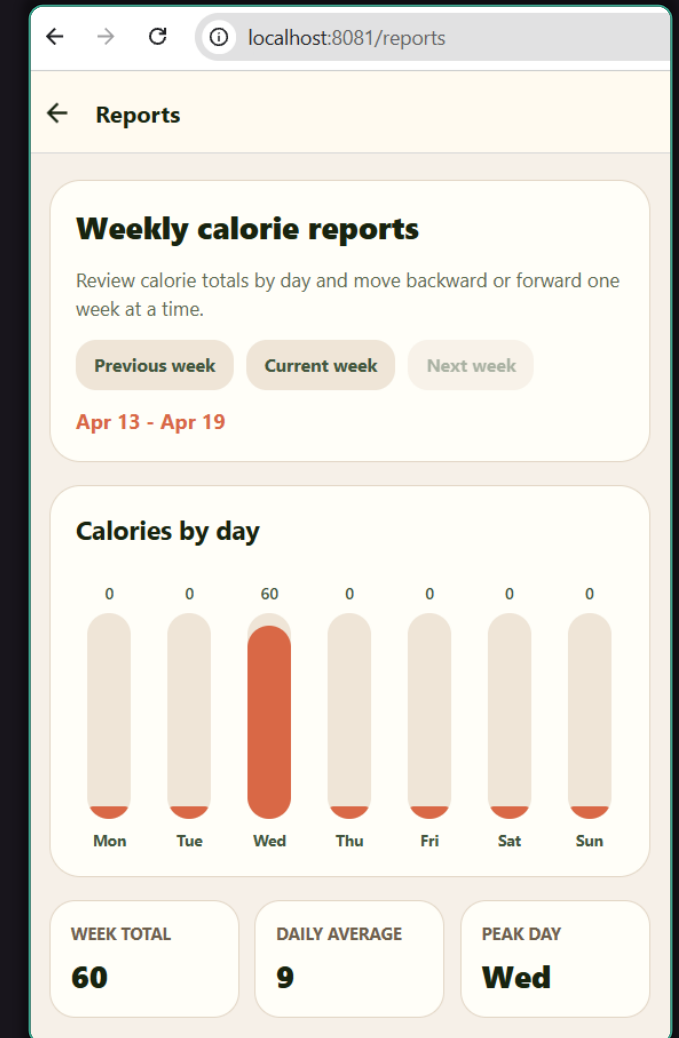
Buttons: Add entry, Cancel



← Edit Food Log

Form fields: pizza, 500, 2026-03-13 15:05

Buttons: Save changes, Cancel

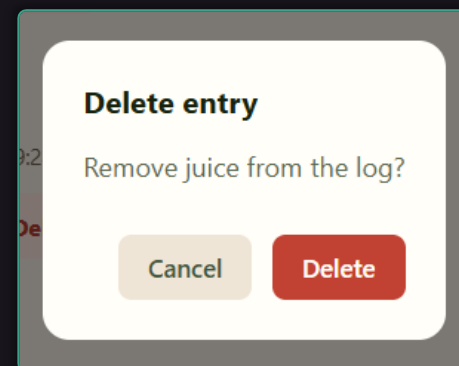


Delete Confirm Screen

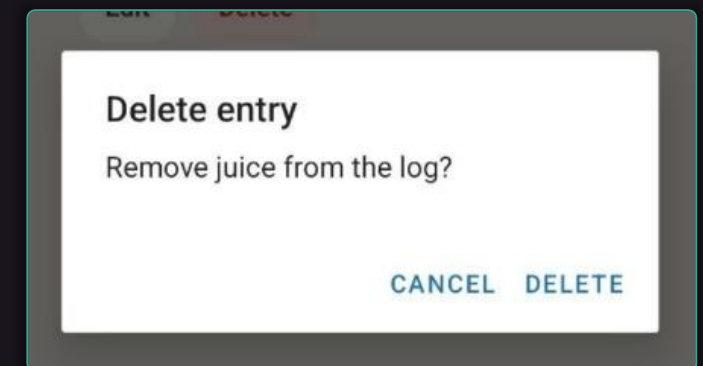
- **Delete confirmation** most probably will be implemented with **Alert.alert** (native popup dialogs)
 - It will **work in iOS / Android**, but will **fail in Web**
- AI prompt to implement "Delete Confirmation" dialog

Implement "**Delete Confirm**" using the device native confirm dialog (on supported platforms). In React Web use a modal popup dialog.

Web (modal)



Android (native)



Data Fetching and Backend

Connecting Expo Apps with Supabase,
Dynamic Lists with FlatList



Connecting an Expo App to Supabase

- **Fetch data** from **Supabase**:

```
const { data, error } = await  
supabase.from("food_logs").select("*");
```

- Always handle **loading** state, **error** state, **empty** state
- **Render data** with **FlatList**:

```
<FlatList data={foodLogs} keyExtractor={(item) =>  
item.id} renderItem={renderFoodLog} ... />
```

- **Mobile data concerns**: slow networks, offline states, retry actions, secure key handling (use **.env**)

MCP + AI Workflow for Supabase

1. Create a **project** in Supabase
2. Connect the **Supabase MCP** to your AI dev agent
3. Run AI prompt to generate the **DB schema** → generate migration → apply migration
 - Optionally, generate **RLS policies**
4. Fill **sample data** in the DB
5. Generate / update **CRUD logic** in the app
 - Optionally, generate **login / register / logout / protected routes**
6. **Run** and **test** the app

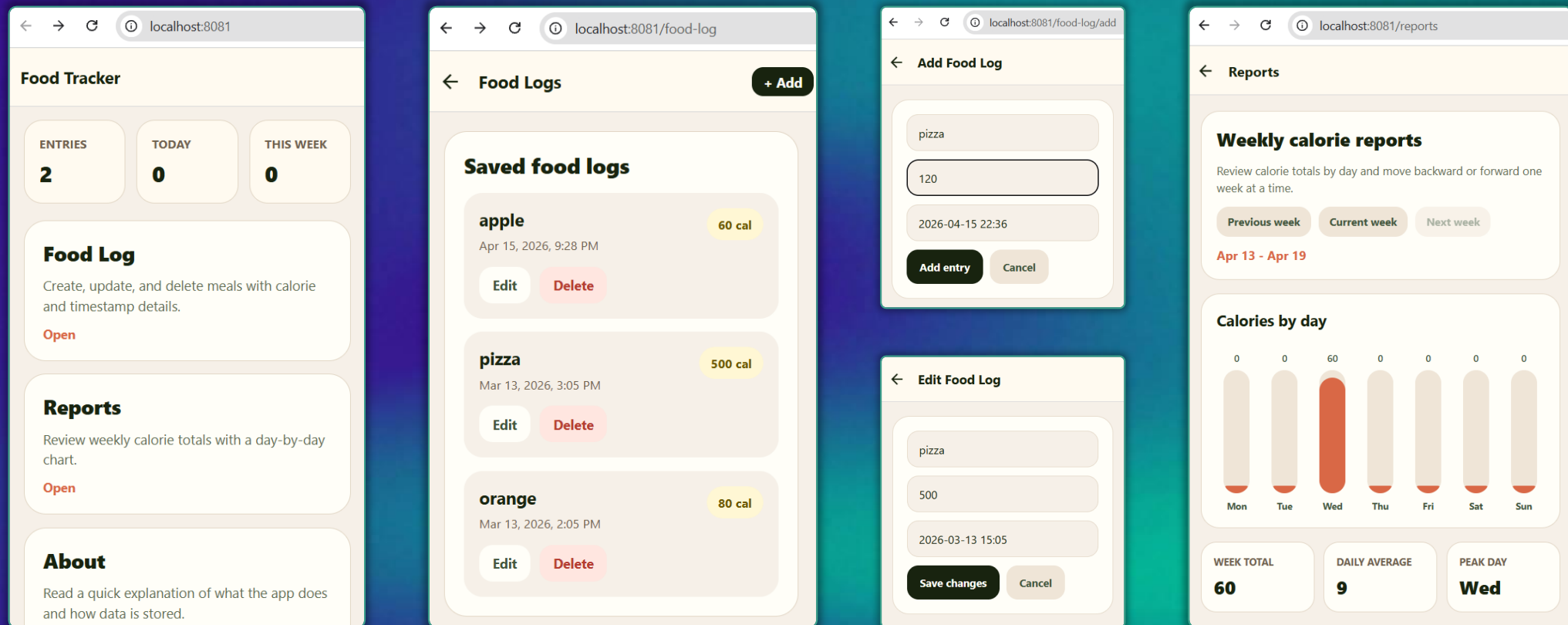
Expo Router Caching and Supabase

- **Expo Router** keeps components alive after navigation (unlike the classical React Router)
 - Once visited, app **screens do not unmount** (stay alive, **cached**, for faster loading when needed again)
- Expo developers should implement **focus-based re-fetch**:

```
useFocusEffect(  
  useCallback(() => {  
    loadData();  
  }, [])  
);
```



Exercise: Food Tracker with Supabase



Exercise: Food Tracker with Supabase

- Create **Supabase** project for the **Food Tracker** app
- Install and connect **Supabase MCP**
- Sample **AI prompt** to implement Supabase backend

Generate **DB migration** to create the required **DB tables**

Fill **sample data** in DB

Replace Async Storage data layer with **Supabase**

backend: create / read / update / delete operations

Keep app **simple**: no users, no auth, no access control

Keep **code clean**, modular and beginner-friendly

Food Logs in Supabase



localhost:8081/food-log

Food Logs

+ Add

Saved food logs

apple

Apr 15, 2026, 9:28 PM

60 cal

EditDelete

pizza

Mar 13, 2026, 3:05 PM

500 cal

EditDelete

orange

Mar 13, 2026, 2:05 PM

80 cal

EditDelete

supabase.com/dashboard/project/xvmhccxqpahnkjsxgnif/editor/17509?schema=public

MyOrg FREE / FoodTracker-old / main PRODUCTION Connect

Feedback

Table Editor

food_logs

Filter by id, food_name, calories... or ask AI

Sort RLS disabled Index Advisor Enable Realtime Insert

id	food_name	calories	consumed_at
209e71f0-2091-4bc4-94d0-26826ce8da2c	apple	60	2026-04-15 18:28:00+00
3f5bf0b4-c1d5-4395-b132-9c22ae0d2223	orange	80	2026-03-13 12:05:00+00
98a727f3-da59-47e3-86df-95ac40b179ed	pizza	500	2026-03-13 13:05:00+00
ca9662f7-2022-4e45-b493-a22fb1c6a594	juice	75	2026-04-15 20:54:00+00

schema public

+ New table

Search tables...

food_logs UNRESTRICTED

food_logs

id uuid

food_name text

calories int4

consumed_at timestamptz

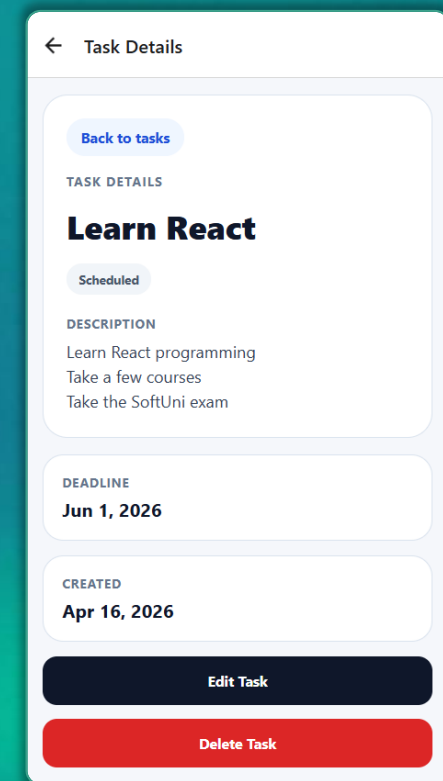
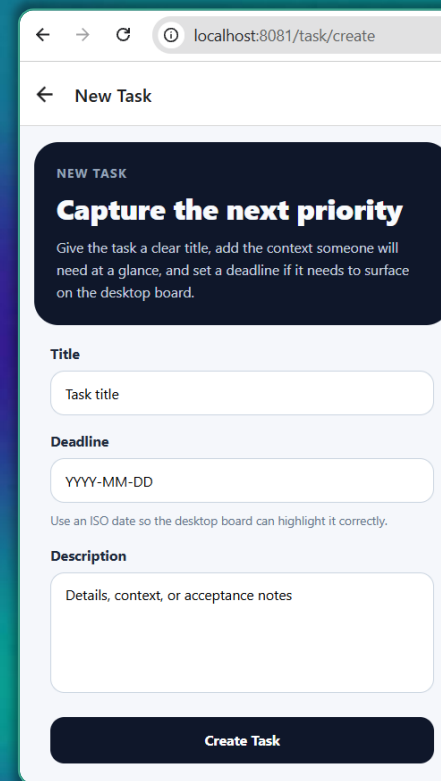
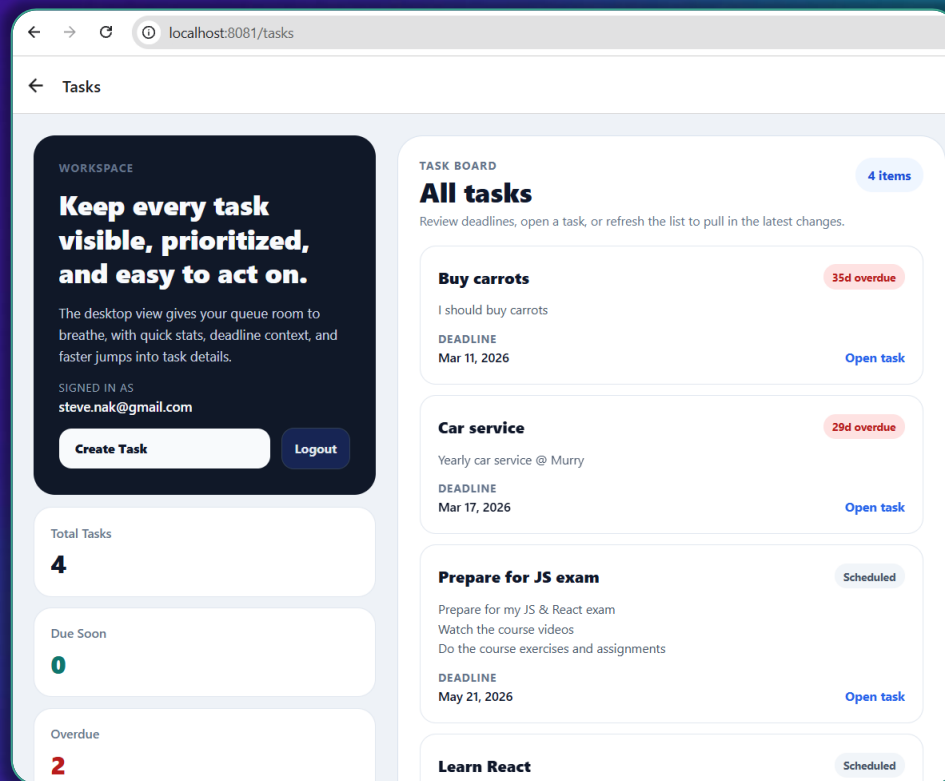
created_at timestamptz

updated_at timestamptz

69

Exercise: Task List Mobile App

Building a Mobile App with Expo + Supabase



The "Task List" App

- Build a simple **task list** mobile app with **Expo + Supabase**
 - **Register / login, view** task list, **create, edit, delete** tasks
 - **Supabase** backend: keep app data in Supabase DB
 - **Responsive layout**: support large and small screens
- Steps to build the app:
 1. Create a new **project in Supabase**
 2. Install **Supabase MCP**
 3. AI prompts to build the **app screens** and **logic**

Exercise: Task List Mobile App

Build a mobile app "**Task List**" with **Expo**, **TypeScript** and **Supabase**

- **Screens**: Home | Tasks | Task Details | Create Task | Edit Task
- Keep app data in **Supabase**: tasks (title, description, deadline)
- Generate and run **DB migrations** to create the required DB tables
- Implement **authentication** (login, register, logout)
- After user registration, insert **2 sample tasks** in the new account
- Implement **create**, **read**, **update**, and **delete** (CRUD) functionality
- Add native UI alerts for **delete confirmations**, with fallbacks for Web (with a modal popup)
- Implement **responsive layout**: render Web UI for large screens and mobile UI for small screens

Screens and Navigation

- **App screens:**

- (main) → Home, Tasks, Task Details, Create Task, Edit Task
- (auth) → Login, Register

- **Routing groups:**

- (auth) group + (main) group

```
▼ app
  > (auth)
  > (main)
  ⚙ _layout.tsx
  ⚙ index.tsx
```

```
▼ (main)
  > task
  ⚙ _layout.tsx
  ⚙ index.tsx
  ⚙ tasks.tsx
```

```
▼ (auth)
  ⚙ _layout.tsx
  ⚙ login.tsx
  ⚙ register.tsx
```

- **Authentication flow:**

```
const { user, logout } = useAuth();
if (!user) return <Redirect href="/(auth)/login" />;
```

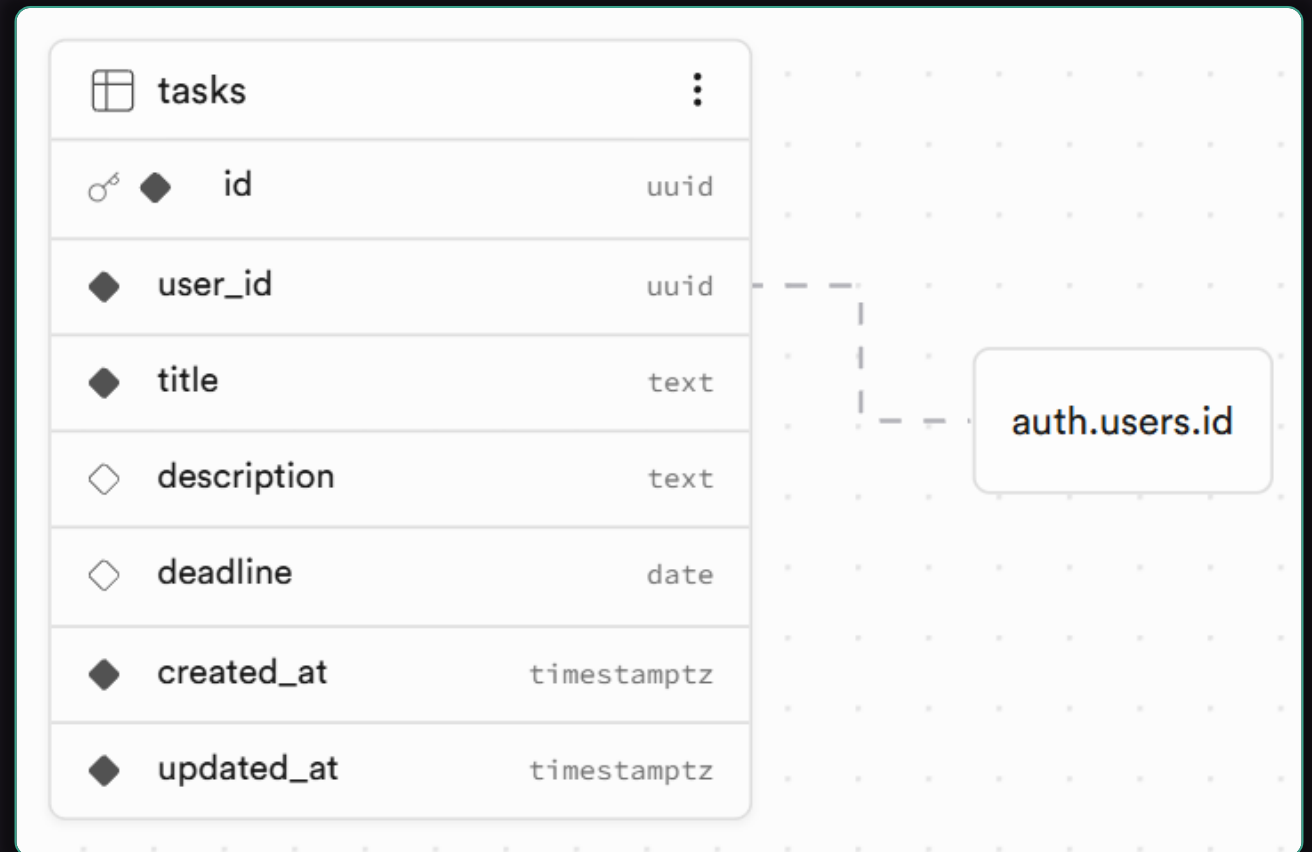
- Register → Login → Persist session → Logout
- Protect task actions for logged-in users

App Data Model

- **Task** data type:

```
export type Task = {  
  id: string;  
  user_id: string;  
  title: string;  
  description: string | null;  
  deadline: string | null;  
  created_at: string;  
};
```

- **DB schema** in Supabase:



App Data in Supabase

tasks

+

🔍

Filter by id, user_id, title... or ask AI

☰ Sort

4 RLS policies

💡 Index Advisor

📍 Enable Realtime

Role postgres ▾

📖

🔄

▼ Insert

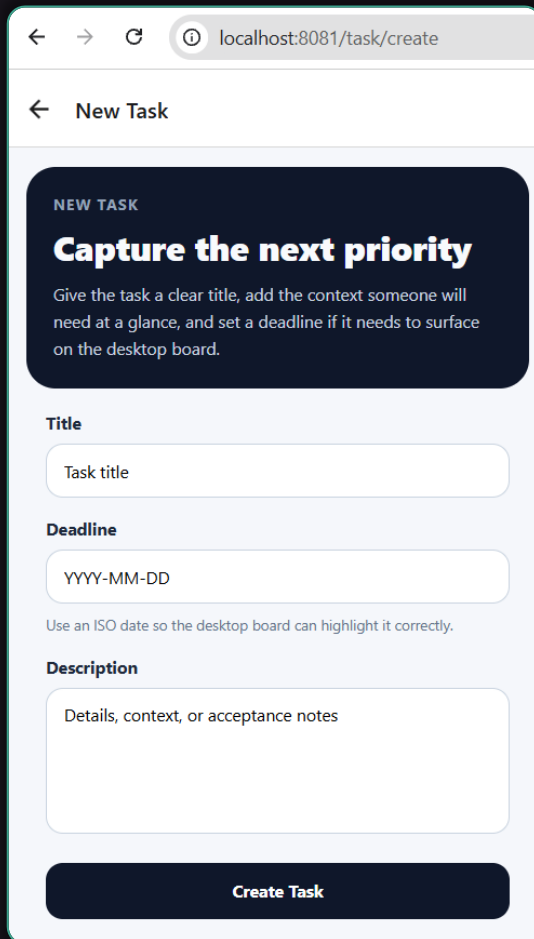
☐	 id uuid ▾	 user_id uuid ▾	title text ▾	description text ▾	deadline date ▾
☐	17c9ecea-a178-4	7c266c00-7cc... →	Prepare for JS exam	Prepare for my JS & React e	2026-05-21
☐	2a246c44-9255-	7c266c00-7cc... →	Learn React	Learn React programming T	2026-06-01
☐	387ae6a8-34bf-4	65d9d03a-ce0... →	Bay Ivan's Task	A task from Bay Ivan	NULL
☐	679a6901-94c1-4	7c266c00-7cc... →	Buy carrots	I should buy carrots	2026-03-11
☐	67de229b-669c-	7c266c00-7cc... →	Car service	Yearly car service @ Murry	2026-03-17

Specific App Features

- **Delete confirmations:**
 - Native mobile: **Alert.alert(...)**
 - Web fallback: **modal popup** dialog
 - Good example of platform-specific branching
- **Responsive layout:**
 - **Small screens:** stacked mobile cards
 - **Large screens / web:** sidebar + details panel
 - One codebase, adaptive layout with screen width checks

Responsive Layout

- **Small screen** layout:
- **Large screen** layout:



localhost:8081/task/create

← New Task

NEW TASK

Capture the next priority

Give the task a clear title, add the context someone will need at a glance, and set a deadline if it needs to surface on the desktop board.

Title

Task title

Deadline

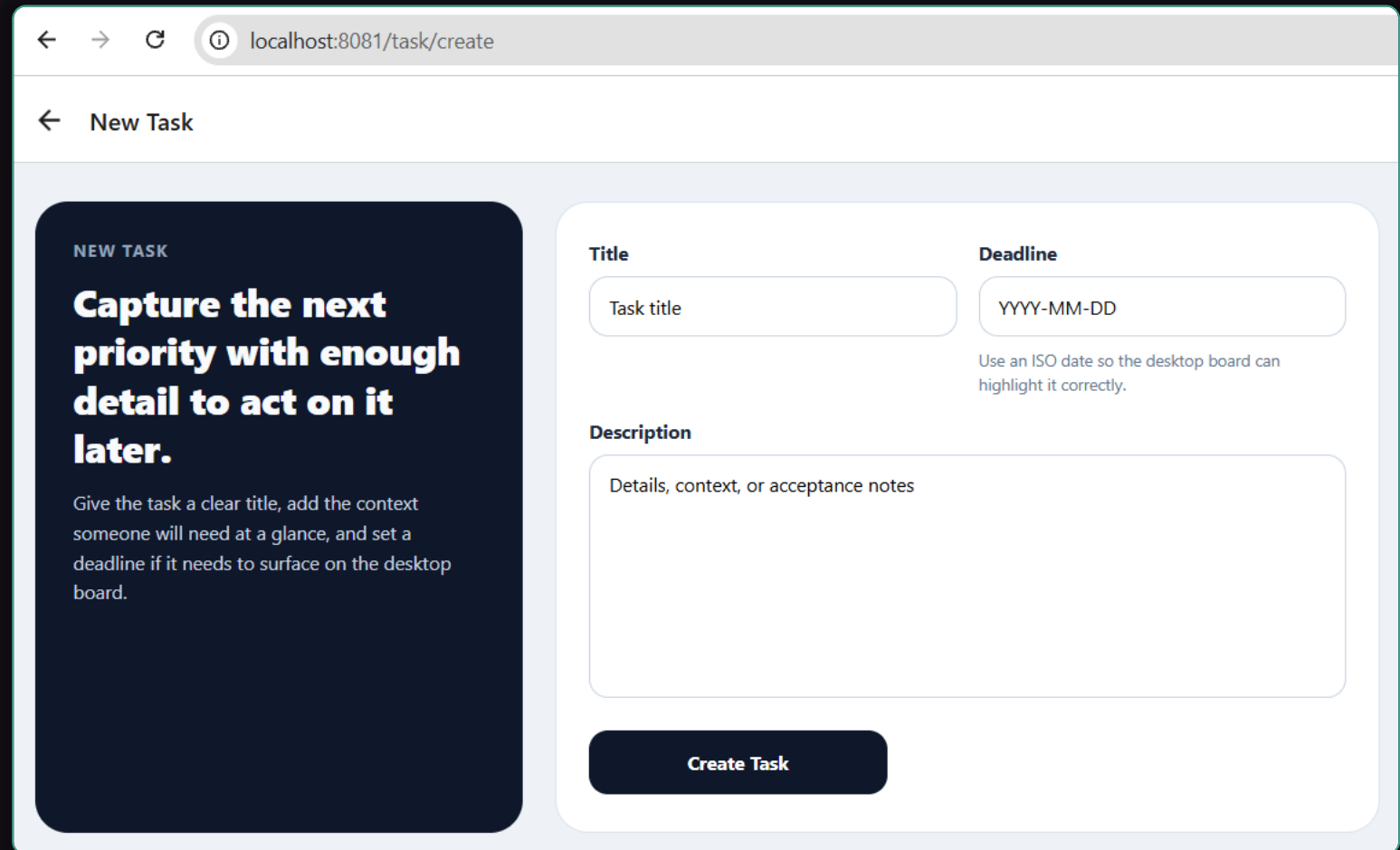
YYYY-MM-DD

Use an ISO date so the desktop board can highlight it correctly.

Description

Details, context, or acceptance notes

Create Task



localhost:8081/task/create

← New Task

NEW TASK

Capture the next priority with enough detail to act on it later.

Give the task a clear title, add the context someone will need at a glance, and set a deadline if it needs to surface on the desktop board.

Title

Task title

Deadline

YYYY-MM-DD

Use an ISO date so the desktop board can highlight it correctly.

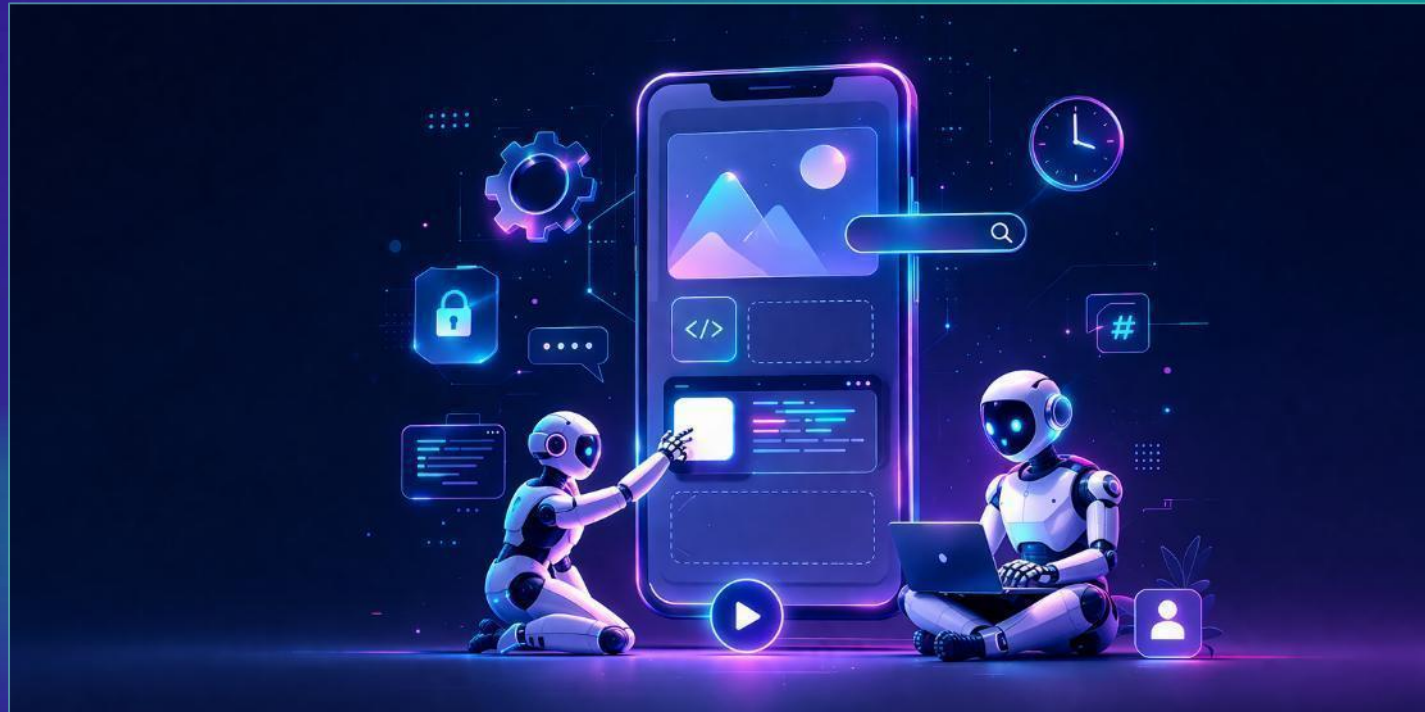
Description

Details, context, or acceptance notes

Create Task

Publishing on the App Store

Android / iOS App Store Concepts, Expo
Application Services, App Metadata



From Dev App to Store App

- **Development mode** vs. **production mode**
 - **Development mode** is not the final product!
 - **App store publishing** needs: app metadata, bundle identifiers, signing/certificates, production builds
- **Expo Application Services (EAS)** simplifies the app store publishing process
 - Build and publish to **Apple App Store**
 - Build and publish to **Google Play App Store**

From Dev App to Store App

- Install **EAS CLI** (Expo Application Services) and **login**:

```
npm install -g eas-cli
```

```
eas login
```

```
PS task-list> eas login
Log in to EAS with email or username (exit and run
eas login --help to see other login options)
✓ Email or username ... [redacted]
✓ Password ... *****
Logged in
```

- Configure the **app metadata** (**app.json**)
 - app name, slug, icons, splash
 - bundle / package identifiers, version

Build and Submit the App

- **Build** the app for **Android** / **iOS** platforms (in Expo cloud):

```
eas build --platform android  
eas build --platform ios
```

```
PS task-list> eas build --platform android
```

```
See logs: https://expo.dev/accounts/nakov/projects/task-list/builds/f8cbd0d6-516e-470f-99e2-421aaaaeee36f
```

```
Waiting for build to complete. You can press Ctrl+C to exit.  
- Build in progress...
```

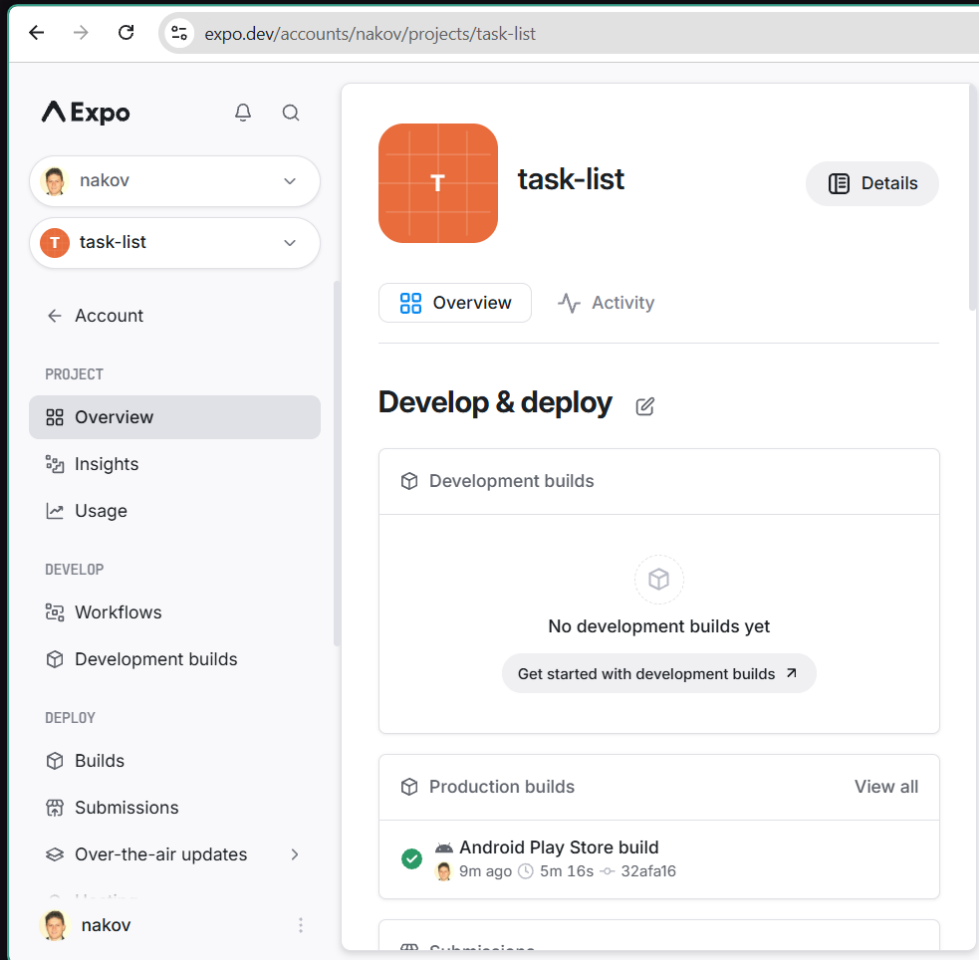
- Submit the app

```
eas submit --platform android  
eas submit --platform ios
```

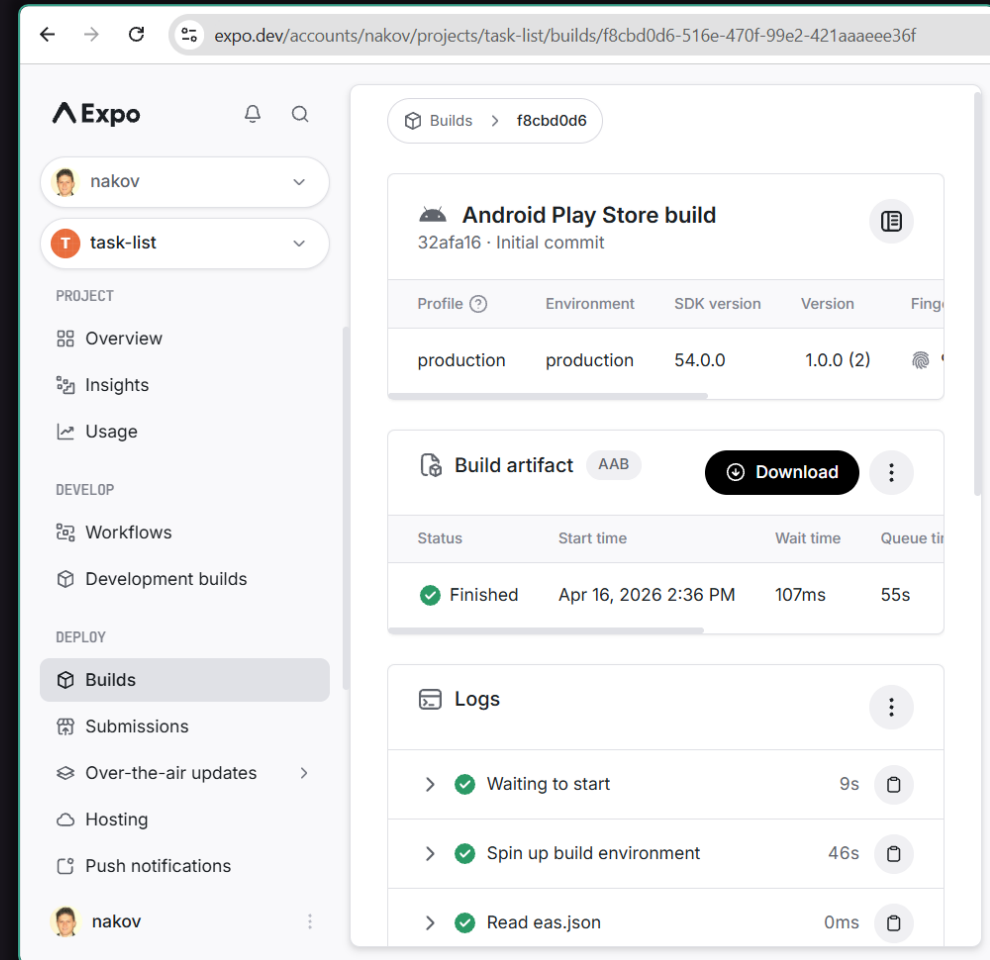
- Store listing needs **review** and **approval** (can take time)
- Test carefully before submission!

Expo Dev Dashboard

- Expo Dev Dashboard – <https://expo.dev>



The screenshot shows the Expo Dev Dashboard for a user named 'nakov'. The left sidebar contains navigation links for Account, PROJECT (Overview, Insights, Usage), DEVELOP (Workflows, Development builds), and DEPLOY (Builds, Submissions, Over-the-air updates). The main content area displays the 'task-list' project with tabs for Overview and Activity. Under the 'Develop & deploy' section, there is a 'Development builds' card indicating 'No development builds yet' and a 'Production builds' card showing a successful 'Android Play Store build' from 9m ago.



The screenshot shows the details of a specific build (f8cbd0d6) on the Expo Dev Dashboard. The left sidebar is similar to the previous view but highlights the 'Builds' section under 'DEPLOY'. The main content area shows the 'Android Play Store build' for the 'task-list' project. It includes a table with build details and a 'Build artifact' section with a 'Download' button. Below, the 'Logs' section shows the build process steps.

Profile	Environment	SDK version	Version	Fingerprint
production	production	54.0.0	1.0.0 (2)	[Fingerprint]

Status	Start time	Wait time	Queue time
Finished	Apr 16, 2026 2:36 PM	107ms	55s

Logs

- Waiting to start (9s)
- Spin up build environment (46s)
- Read eas.json (0ms)

Lesson Summary

- **React Native, Expo and Expo Go:** build mobile apps in **React + JavaScript / TypeScript**
- Running Expo projects: **npm run web**
 - Run in **Web browser** or **in Expo Go** (open with QR code)
- Use the same React mental model: TSX components, states, hooks, etc.
- **Expo Router:** implements **screens** and **navigation**
- **Expo + Supabase:** the same as React + Supabase



Questions?



Postbank – Exclusive Partner for SoftUni AI



- One of the leading **banking institutions** in Bulgaria
- Member of the Eurobank Group with € 103 billion assets
- Innovative trendsetter with next generation **beyond banking**, transforming today, empowering tomorrow
- Certified **Top Employer 2025** by the international Top Employers Institute
- Proven people care and **wellbeing initiatives**
- Benefits and unlimited access to professional, **personal and leadership trainings and programs**
- www.postbank.bg / careers.postbank.bg



Diamond Partners of Software University



SUPER
HOSTING
.BG

encorp.ai

INDEAVR
Serving the high achievers

createX

DRAFT
KINGS
THE CROWN IS YOURS

Diamond Partners of SoftUni Digital



**SUPER
HOSTING
.BG**

 encorp.ai



INDEAVR
Serving the high achievers



Diamond Partners of SoftUni Digital



HUMAN

NETPEAK
DIGITAL GROWTH PARTNER



MagmaLAB | E-комерс агенция

ETIENYANEV
Break Your *Limits*. Live Your *Brand*.

1FOR FIT



Diamond Partners of SoftUni Creative



**SUPER
HOSTING
.BG**

 encorp.ai

INDEAVR
Serving the high achievers

createX

 **DRAFT
KINGS**
THE CROWN IS YOURS

Organization Partners of SoftUni Creative



Studio π
Think Big • Design Smart

THE
BUCKS
TOWN'S
WORK

